


<http://cialdea.dia.uniroma3.it/teaching/pf/>

Linguaggi funzionali

- di alto livello: un programma è una funzione.
- di tipo dichiarativo: il programmatore specifica **che cosa** calcola la funzione, piuttosto che **come**.

Objective Caml

- Linguaggio della famiglia ML sviluppato e distribuito dall'INRIA (Francia) dal 1984
- Supporta diversi stili di programmazione: funzionale, imperativo, a oggetti.

Paradigmi di programmazione

Diverse tipologie di linguaggi di programmazione.

Sotto ogni linguaggio c'è un **modello di calcolo**, che determina

- le operazioni eseguibili
- una classe di linguaggi
- uno stile di programmazione (“paradigma” di programmazione)

Modelli astratti e modelli rappresentati dall'hardware

Linguaggi dichiarativi

Un programma è più vicino alla descrizione di

- **che cosa** si deve calcolare,
- piuttosto che a **come** calcolare
(linguaggi **imperativi**: assegna il valore ... alla variabile x, poi entra in un ciclo fino a che ..., fai questo e quello, ... alla fine riporta il valore della variabile ...).

Esempio: calcolo del fattoriale

Cos'è il fattoriale di un numero naturale?

$$\begin{aligned} n! &= 1 \times 2 \times \dots \times n-1 \times n \\ &= (n-1)! \times n \end{aligned}$$

Caso particolare:

$$0! = 1$$

```
(* fact: int -> int *)  
let rec fact n =  
  if n=0  
  then 1  
  else fact (n-1) * n
```

COMANDI, ASSEGNAZIONE, CICLI

I costrutti di controllo fondamentali sono

- applicazione di funzioni : $f(x)$ o semplicemente $f\ x$
- composizione di funzioni : $f\ (g\ x)$
- ricorsione

Modalità interattiva: ciclo READ - EVAL - PRINT

```
# pred 6;;
- : int = 5
# abs 7;;
- : int = 7
# abs (-7);;
- : int = 7
# 6 * 7;;
- : int = 42

# let numero = 6*7;;
val numero : int = 42
# succ (pred (pred numero));;
- : int = 41
# let rec fact n =
  if n=0 then 1
  else n * fact(n-1);;
  val fact : int -> int = <fun>
# fact 4;;
- : int = 24
```

- READ: il compilatore legge un'espressione o una dichiarazione
- EVAL: calcola il valore dell'espressione e ne deduce il tipo (o "memorizza" la dichiarazione)
- PRINT: stampa il tipo e il valore dell'espressione (o della variabile "dichiarata")

Inferenza di tipi

OCaml può dedurre (inferire) qual è il tipo di un'espressione senza bisogno di dichiarazioni esplicite

```
# if numero > 0 then "pippo" else "pluto";;  
- : string = "pippo"  
# let double n = 2 * n;;  
val double : int -> int = <fun>
```

OCaml è un linguaggio **a tipizzazione statica**: ogni espressione ha un tipo che può essere **determinato a tempo di compilazione**.

Tipi semplici: bool, int, float, string, char, unit, exn

Prodotto cartesiano (tipo delle coppie): $T_1 \times T_2$

Tipo delle funzioni: $\text{Dominio} \rightarrow \text{Codominio}$

Espressioni

- I costrutti di base sono **espressioni** (non comandi)
- Le espressioni **hanno sempre un valore e un tipo**
- Il calcolo procede **valutando espressioni**, non ci sono effetti collaterali
Valutare un'espressione = semplificarla fino ad ottenere un'espressione non più semplificabile, cioè un valore.

```
# double((3+7) * (32 mod 5));  
- : int = 40
```

```
double((3+7) * (32 mod 5))  
  ⇒ double (10 * (32 mod 5))  
  ⇒ double (10 * 2)  
  ⇒ double 20  
  ⇒ 40
```

Dichiarazioni: sintassi

```
let numero = 6*7
```

Forma generale di una dichiarazione

let <identificatore> = <espressione>

Dichiarazione di funzione:

```
let double = function n -> 2 * n
```

O anche, con una sintassi più familiare:

```
let double n = 2 * n
```

let <identificatore> <parametri> = <espressione>

Per dichiarare funzioni ricorsive:

let rec <identificatore> <parametri> = <espressione>

Ambiente di valutazione

Un ambiente è una collezione di **legami** variabile-valore:

var_n	val_n
var_{n-1}	val_{n-1}
...	...
var_1	val_1

Ambiente iniziale:

...	...
<i>not</i>	<i>function x</i> $\rightarrow$...
...	...
<i>mod</i>	<i>function x</i> $\rightarrow$ <i>function y</i> $\rightarrow$...
...	...

} Ambiente del
modulo
Stdlib

Estensione dell'ambiente mediante dichiarazioni

La valutazione di una dichiarazione aggiunge un nuovo legame all'ambiente:

```
# let two = 2;;  
val two : int = 2
```

<i>two</i>	2
<i>ambiente del modulo Stdlib</i>	

```
# let three = two + 1;;  
val three : int = 3
```

- Viene valutata l'espressione `two + 1` nell'ambiente esistente, dove il valore di `two` è 2;
- Viene creato un nuovo legame, di `three` con il valore di `two + 1`

<i>three</i>	3
<i>two</i>	2
<i>ambiente del modulo Stdlib</i>	

Valore di una variabile in un ambiente

L'ambiente viene gestito come una **pila**: i nuovi legami vengono aggiunti in alto e si cerca il valore delle variabili a partire dall'alto.

```
# let two = "due";;  
val two : string = "due"
```

<i>two</i>	"due"
<i>three</i>	3
<i>two</i>	2
<i>ambiente del modulo Stdlib</i>	

In questo ambiente, il valore di `two` è "due": il nuovo legame nasconde il vecchio.

Scopo statico

Il valore delle variabili (globali) in una dichiarazione viene determinato a tempo di compilazione

six	6
ambiente del modulo Stdlib	

```
# let six=6;;
```

Scopo statico

Il valore delle variabili (globali) in una dichiarazione viene determinato a tempo di compilazione

sixtimes	function n -> six * n
six	6
ambiente del modulo Stdlib	

```
# let six=6;;  
# let sixtimes n=  
  six*n;;
```

Scopo statico

Il valore delle variabili (globali) in una dichiarazione viene determinato a tempo di compilazione

six	50
sixtimes	function n -> six * n
six	6
ambiente del modulo Stdlib	

```
# let six=6;;  
# let sixtimes n=  
  six*n;;  
# let six=50;;
```

Scopo statico

Il valore delle variabili (globali) in una dichiarazione viene determinato al tempo di compilazione

six	50
sixtimes	function n -> six * n
six	6
ambiente del modulo Stdlib	

```
# let six=6;;  
# let sixtimes n=  
  six*n;;  
# let six=50;;  
# six;;  
- : int = 50  
# sixtimes 3;;  
- : int = 18
```

Il valore di `six` nel corpo di `sixtimes` viene cercato **nell'ambiente di dichiarazione** di `sixtimes`, cioè nell'ambiente in cui `sixtimes` è stata definita.

Chiamata di funzioni

sixtimes	function n -> six * n
six	6
...	...

```
# let six=6;;  
# let sixtimes n = six * n;;  
# sixtimes (2+1);;
```

Quando la funzione `sixtimes` viene applicata a un argomento:

- il suo argomento viene valutato nell'ambiente

Chiamata di funzioni

n	3
sixtimes	function n -> six * n
six	6
...	...

```
# let six=6;;  
# let sixtimes n = six * n;;  
# sixtimes (2+1);;
```

Quando la funzione `sixtimes` viene applicata a un argomento:

- il suo argomento viene valutato nell'ambiente
- viene creato un nuovo legame **provvisorio** del parametro formale `n` con il valore dell'argomento
- in questo nuovo ambiente viene valutato il corpo della funzione, `six * n`

Chiamata di funzioni

sixtimes	function n -> six * n
six	6
...	...

```
# let six=6;;  
# let sixtimes n = six * n;;  
# sixtimes (2+1);;  
- : int = 18
```

Quando la funzione `sixtimes` viene applicata a un argomento:

- il suo argomento viene valutato nell'ambiente
- viene creato un nuovo legame **provvisorio** del parametro formale `n` con il valore dell'argomento
- in questo nuovo ambiente viene valutato il corpo della funzione, `six * n`
- il legame provvisorio viene cancellato.

Polimorfismo

```
# let first (x,y) = x;;  
val first : 'a * 'b -> 'a = <fun>  
# first (true,0);;  
- : bool = true  
# first(0,"pippo");;  
- : int = 0
```

- 'a e 'b sono **variabili di tipo**
(a mano usiamo le lettere greche $\alpha, \beta, \dots$)
- first si può applicare a qualsiasi coppia, ha un numero infinito di tipi
- Tipo di “qualsiasi coppia”: $\alpha \times \beta$ ('a * 'b)

```
# let id x = x;;  
val id : 'a -> 'a = <fun>  
# id 6.3;;  
- : float = 6.3  
# id "pluto";;  
- : string = "pluto"
```

Le funzioni sono oggetti di prima classe

```
# double;;  
- : int -> int = <fun>
```

- `double` è una funzione

Le funzioni sono oggetti di prima classe

```
# double;;  
- : int -> int = <fun>  
# (double, "pippo");;  
- : (int -> int) * string = (<fun>, "pippo")
```

- `double` è una funzione
- Le funzioni possono essere **componenti di una struttura dati**

Le funzioni sono oggetti di prima classe

```
# double;;  
- : int -> int = <fun>  
# (double, "pippo");;  
- : (int -> int) * string = (<fun>, "pippo")  
# first(double, "pippo");;  
- : int -> int = <fun>
```

- `double` è una funzione
- Le funzioni possono essere **componenti di una struttura dati**
- Le funzioni possono essere **argomenti** di altre funzioni

Le funzioni sono oggetti di prima classe

```
# double;;  
- : int -> int = <fun>  
# (double, "pippo");;  
- : (int -> int) * string = (<fun>, "pippo")  
# first(double, "pippo");;  
- : int -> int = <fun>  
# let times n = function m -> n * m;;  
val times : int -> int -> int = <fun>
```

- `double` è una funzione
- Le funzioni possono essere **componenti di una struttura dati**
- Le funzioni possono essere **argomenti** di altre funzioni
- Le funzioni possono essere **valori** riportati da altre funzioni

Le funzioni sono oggetti di prima classe

```
# double;;  
- : int -> int = <fun>  
# (double, "pippo");;  
- : (int -> int) * string = (<fun>, "pippo")  
# first(double, "pippo");;  
- : int -> int = <fun>  
# let times n = function m -> n * m;;  
val times : int -> int -> int = <fun>  
# let double = times 2;;  
val double : int -> int = <fun>  
# (times 3) 5;;  
- : int = 15
```

- `double` è una funzione
- Le funzioni possono essere **componenti di una struttura dati**
- Le funzioni possono essere **argomenti** di altre funzioni
- Le funzioni possono essere **valori** riportati da altre funzioni

Uso delle parentesi

- Nei tipi (**espressioni di tipo**)

`int -> int -> int = int -> (int -> int)`

si associa a destra

- Nelle **espressioni**

`(times 3) 5 = times 3 5`

si associa a sinistra

```
# let square n = n* n;;  
val square : int -> int = <fun>  
# double square 3;;  
Characters 0-6:  
  double square 3;;  
  ^^^^^
```

Error: This function is applied to too many arguments;
maybe you forgot a `;`

```
# double (square 3);;  
- : int = 18
```

Funzioni di ordine superiore

Prendono come argomento o riportano come valore una funzione

```
(* sum : (int -> int) -> (int * int) -> int *)
(* sum f (lower,upper) = (f lower) + (f (lower+1)) +
                           ... + (f upper) *)

let rec sum f = function
  (lower,upper) -> if lower > upper then 0
                   else f lower + sum f (lower +1,upper)
```

O anche

```
let rec sum f (lower,upper) =
  if lower > upper then 0
  else f lower + sum f (lower +1,upper)
```

Tipo di sum

```
sum:   (int -> int) -> ((int * int) -> int)
sum double:   (int * int) -> int      sommatoria dei doppi
sum double (3,5):   int                6 + 8 + 10 = 24
```

Sommatoria in “forma currificata”

La funzione `sum` può essere applicata anche soltanto al suo primo argomento:

```
sum double: (int * int) -> int
```

Vogliamo “risparmiare” ancora qualche parentesi e fare in modo che `sum` si possa applicare alla funzione e all'estremo inferiore soltanto?

```
(* sum : (int -> int) -> int -> int -> int *)  
let rec sum f lower upper =  
  if lower > upper then 0  
  else f lower + sum f (lower +1) upper  
  
# sum double;;  
- : int -> int -> int = <fun>  
# sum double 3;;  
- : int -> int = <fun>
```

Funzioni in forma “currificata”

```
(* mult:  
   int * int -> int *)  
let mult (m,n) = m * n
```

```
(* times:  
   int -> int -> int *)  
let times m n = m * n
```

times è la **forma currificata** di **mult**: calcola gli stessi valori, ma “consuma un argomento alla volta”

```
let times n = function m -> n*m  
let times = function n -> function m -> n*m
```

Funzioni in forma “currificata”

`(* mult: int * int -> int *)`
`let mult (m,n) = m * n`
`(* times: int -> int -> int *)`
`let times m n = m * n`

times è la **forma currificata** di **mult**: calcola gli stessi valori, ma “consuma un argomento alla volta”

```
let times n = function m -> n*m
let times = function n -> function m -> n*m
```

Può anche essere applicata solo parzialmente:

times 5: `int -> int` è un'**applicazione parziale** di **times**.

$$\text{sum (times 5) 1 10} = \sum_{k=1}^{10} (5 \times k)$$

Funzioni in forma “currificata”

```
(* mult:                                     (* times:
   int * int -> int *)                       int -> int -> int *)
let mult (m,n) = m * n                      let times m n = m * n
```

times è la **forma currificata** di **mult**: calcola gli stessi valori, ma “consuma un argomento alla volta”

```
let times n = function m -> n*m
let times = function n -> function m -> n*m
```

Può anche essere applicata solo parzialmente:

times 5: int -> int è un’**applicazione parziale** di **times**.

$$\text{sum (times 5) 1 10} = \sum_{k=1}^{10} (5 \times k)$$

```
# List.map (times 2) [1; 2; 3; 4];;
- : int list = [2; 4; 6; 8]
```

Forma currificata di una funzione su tuple

In generale, f_c è la forma currificata di f se

$$\begin{aligned} f &: t_1 \times \dots \times t_n \rightarrow t \\ f_c &: t_1 \rightarrow (t_2 \rightarrow \dots \rightarrow (t_n \rightarrow t) \dots) \end{aligned}$$

e per ogni $a_1, \dots, a_n$: $f(a_1, \dots, a_n) = (((f_c a_1) a_2) \dots a_n)$

Le parentesi possono essere omesse

- sia nel tipo di f_c (si associa a destra),
- sia nell'applicazione di f_c (si associa a sinistra).

Forma currificata di una funzione su tuple

In generale, f_c è la forma currificata di f se

$$\begin{aligned} f &: t_1 \times \dots \times t_n \rightarrow t \\ f_c &: t_1 \rightarrow (t_2 \rightarrow \dots \rightarrow (t_n \rightarrow t) \dots) \end{aligned}$$

e per ogni $a_1, \dots, a_n$: $f(a_1, \dots, a_n) = (((f_c a_1) a_2) \dots a_n)$

Le parentesi possono essere omesse

- sia nel tipo di f_c (si associa a destra),
- sia nell'applicazione di f_c (si associa a sinistra).

Espressioni per denotare funzioni

- 1 variabili: **times**
- 2 astrazioni funzionali: **function x -> function y -> x*y**,
o anche **fun x y -> x*y**
- 3 espressioni funzionali: **sum (times 5), sum (times 5) 0**

Funzioni in forma “currificata” (II)

```
# let pair x y = (x,y);;  
val pair : 'a -> 'b -> 'a * 'b = <fun>
```

pair 3: 'a -> int * 'a

è la funzione che “accoppia 3” al suo argomento

```
# let greaterthan x y = y > x;;  
val greaterthan : 'a -> 'a -> bool = <fun>
```

greaterthan 0: int -> int

è un predicato: essere maggiore di 0

```
# List.filter (greaterthan 0) [2; 0; -1; 4; -8; -10; 5];;  
- : int list = [2; 4; 5]
```

Funzioni in forma “currificata” (III)

```
(* sumbetween : int * int -> int *)  
(* sumbetween (n,m) = n + (n+1) + ... + m *)
```

Funzioni in forma “currificata” (III)

```
(* sumbetween : int * int -> int *)  
(* sumbetween (n,m) = n + (n+1) + ... + m *)  
let rec sumbetween (n,m) =  
  if n>m then 0 else n + sumbetween (n+1,m)
```

Funzioni in forma “currificata” (III)

```
(* sumbetween : int * int -> int *)  
(* sumbetween (n,m) = n + (n+1) + ... + m *)  
let rec sumbetween (n,m) =  
  if n>m then 0 else n + sumbetween (n+1,m)
```

```
(* sbt : int -> int -> int *)  
(* sbt n m = sumbetween (n,m) *)  
let rec sbt n m =  
  if n>m then 0 else n + sbt (n+1) m
```

sbt “calcola” gli stessi valori di sumbetween, ma “consumando” gli argomenti uno alla volta

```
sumbetween: int * int -> int  
sbt: int -> int -> int
```

sbt è la **forma currificata** di sumbetween,
si può applicare **parzialmente**.

Ad esempio, `sbt 0 : int -> int`, applicata a `n`, riporta la somma dei primi `n` numeri naturali.

Operazioni predefinite in Ocaml

Molte operazioni predefinite in Ocaml sono in forma currificata

```
# max;;  
- : 'a -> 'a -> 'a = <fun>
```

Le operazioni infisse predefinite sono in forma currificata:

```
# (+);;  
- : int -> int -> int = <fun>  
# (+) 3 5;;  
- : int = 8
```

(+) è l'operatore somma usato in forma infissa

```
# ( * );;  
- : int -> int -> int = <fun>  
# (mod);;  
- : int -> int -> int = <fun>  
# (=);;  
- : 'a -> 'a -> bool = <fun>
```

Definizione di operatori infissi

```
# let (++) x y = 2 * (x+y);;  
val ( ++ ) : int -> int -> int = <fun>
```

```
# (++) 3 5;;  
- : int = 16
```

```
# 3 ++ 5;;  
- : int = 16
```

```
# ++ ;;
```

Characters 0-2:

```
++ ;;  
^^
```

Error: Syntax error

```
# (++);;  
- : int -> int -> int = <fun>
```

Composizione di funzioni

```
(* comp : ('a -> 'b) -> ('c -> 'a) -> ('c -> 'b) *)  
(* comp f g = composizione di f con g *)  
let comp f g = function x -> f (g x)
```

O anche

```
let comp f g x = f (g x)
```

Che è come dire

```
comp = function f -> function g -> function x -> f (g x)
```

Esempio: composizione di `double` con la funzione predefinita

`succ: int -> int` (`succ n = n+1`) – e viceversa

```
# comp double succ;;  
- : int -> int = <fun>  
# comp double succ 4;;
```

Composizione di funzioni

```
(* comp : ('a -> 'b) -> ('c -> 'a) -> ('c -> 'b) *)  
(* comp f g = composizione di f con g *)  
let comp f g = function x -> f (g x)
```

O anche

```
let comp f g x = f (g x)
```

Che è come dire

```
comp = function f -> function g -> function x -> f (g x)
```

Esempio: composizione di `double` con la funzione predefinita

`succ: int -> int` (`succ n = n+1`) – e viceversa

```
# comp double succ;;  
- : int -> int = <fun>  
# comp double succ 4;;  
- : int = 10  
# comp succ double 4;;
```

Composizione di funzioni

```
(* comp : ('a -> 'b) -> ('c -> 'a) -> ('c -> 'b) *)  
(* comp f g = composizione di f con g *)  
let comp f g = function x -> f (g x)
```

O anche

```
let comp f g x = f (g x)
```

Che è come dire

```
comp = function f -> function g -> function x -> f (g x)
```

Esempio: composizione di `double` con la funzione predefinita

`succ: int -> int` (`succ n = n+1`) – e viceversa

```
# comp double succ;;  
- : int -> int = <fun>  
# comp double succ 4;;  
- : int = 10  
# comp succ double 4;;  
- : int = 9
```

La composizione definita come operatore infisso

```
# let (@@) f g x = f(g x);;  
val @@ : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>  
  
# let treble n = 3 * n;;  
val treble : int -> int = <fun>  
  
# let sixtimes = double @@ treble;;  
val sixtimes : int -> int = <fun>  
  
# let f = (times 2) @@ ((+) 100);;  
val f : int -> int = <fun>  
  
# f 3;;
```

La composizione definita come operatore infisso

```
# let (@@) f g x = f(g x);;
val @@ : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>

# let treble n = 3 * n;;
val treble : int -> int = <fun>

# let sixtimes = double @@ treble;;
val sixtimes : int -> int = <fun>

# let f = (times 2) @@ ((+) 100);;
val f : int -> int = <fun>

# f 3;;
- : int = 206
```

Tipi predefiniti: bool

Un tipo è un **insieme di valori**

Tipi semplici:

<http://caml.inria.fr/pub/docs/manual-ocaml/core.html#sec473>

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/Stdlib.html>

- `bool`: $\{true, false\}$

Operazioni booleane: `not`, `&&`, `||`

```
# (true && not false) || false;;
```

```
- : bool = true
```

	E' è valutato solo se
$E \ \&\& \ E'$	E ha valore true
$E \ \text{or} \ E'$	E ha valore false

Tipi predefiniti: tipi numerici

- `int`: $\{0, 1, -1, 2, -2, \dots, \text{max_int}, \text{min_int}\}$

Operazioni: `+` `-` `*` `/` `mod` `succ` `pred`

- `float`: floating-point numbers (`0.01`, `3.0`, `-4.0`, `7E-5`, ...)

Operazioni: `+. -.` `*. /. **` `sqrt` `sin` ...

Attenzione: in OCaml non c'è conversione automatica dei tipi numerici

```
(* average : float -> float -> float *)
```

```
(* average x y = media aritmetica di x e y *)
```

```
let average x y = (x *. y) /. 2.0
```

```
# average 2 3;;
```

Tipi predefiniti: tipi numerici

- `int`: $\{0, 1, -1, 2, -2, \dots, \text{max_int}, \text{min_int}\}$

Operazioni: `+` `-` `*` `/` `mod` `succ` `pred`

- `float`: floating-point numbers (`0.01`, `3.0`, `-4.0`, `7E-5`, ...)

Operazioni: `+. -.` `*. /. **` `sqrt` `sin` ...

Attenzione: in OCaml non c'è conversione automatica dei tipi numerici

```
(* average : float -> float -> float *)
(* average x y = media aritmetica di x e y *)
let average x y = (x *. y) /. 2.0
```

```
# average 2 3;;
```

```
Characters 8-9:
```

```
  average 2 3;;
```

^

Error: This expression has type `int` but an expression
was expected of type `float`

Tipi predefiniti: char e string

- **char:** 'a', '9', ' ', ...

Operazioni: `int_of_char: char -> int`,
`char_of_int: int -> char`

```
# int_of_char 'A';;  
- : int = 65  
# char_of_int 65;;  
- : char = 'A'
```

- **string:** "pippo", "pluto", "12Ev", ...

Operazioni: `^` (concatenazione)

```
# "programmazione " ^ "funzionale";;  
- : string = "programmazione funzionale"  
# "ABCDEFGH".[2];;  
- : char = 'C'  
# string_of_int 45;;  
- : string = "45"
```

Operatori di confronto

Uguaglianza, disuguaglianza e operatori di confronto sono **definiti su qualsiasi tipo**, **eccetto le funzioni** e i tipi contenenti funzioni.

```
# 3*8 = 24;;  
- : bool = true  
# "pippo" = "pi" ^ "ppo";;  
- : bool = true  
# true = not true;;  
- : bool = false  
# true <> false;;  
- : bool = true
```

```
# 3*8 <= 30;;  
- : bool = true  
# 6.0 < 5.9;;  
- : bool = false  
# 'A' >= 'B';;  
- : bool = false  
# "abc" > "ABC";;  
- : bool = true  
# false < true;;  
- : bool = true
```

```
# (4,true) <= (10,false);;  
- : bool = true
```

```
# double = times 2;;
```

Exception: Invalid_argument "equal: functional value"

Espressioni condizionali

if E then F else G

è un'espressione condizionale se:

- E è di tipo bool
 - F e G hanno **uno** stesso tipo (almeno un sottotipo in comune)
- ML è un linguaggio fortemente tipato:** il tipo di "if E then F else G" deve essere determinabile a tempo di compilazione

Le espressioni **hanno sempre un tipo e un valore:**

- Il tipo di "if E then F else G" è il tipo **più generale** che F e G hanno in comune
- Il suo valore è:
 - il valore di F se E ha valore true
 - il valore di G se E ha valore false

È un'espressione, non un costrutto di controllo
La parte else non può mancare

Nel valutare un'espressione "if E then F else G":

- se E è true, G non viene valutata
- se E è false, F non viene valutata

Operatori booleani e espressioni condizionali

Un'espressione della forma	equivale a
<code>E && F</code>	<code>if E then F else false</code>
<code>E or F</code>	<code>if E then true else F</code>

Espressioni condizionali che hanno true/false in uno dei due rami (then/else), si possono riscrivere usando gli operatori booleani.

- se uno dei due rami ha true: qual e' l'operatore logico al quale basta valutare un argomento per riportare true?
- se uno dei due rami ha false: qual e' l'operatore a cui basta valutare un argomento per riportare false?

`if E then false else F` $\Rightarrow$

Operatori booleani e espressioni condizionali

Un'espressione della forma	equivale a
$E \ \&\& \ F$	<code>if E then F else false</code>
$E \ \text{or} \ F$	<code>if E then true else F</code>

Espressioni condizionali che hanno true/false in uno dei due rami (then/else), si possono riscrivere usando gli operatori booleani.

- se uno dei due rami ha true: qual e' l'operatore logico al quale basta valutare un argomento per riportare true?
- se uno dei due rami ha false: qual e' l'operatore a cui basta valutare un argomento per riportare false?

$\text{if } E \text{ then false else } F \Rightarrow \text{not } E \ \&\& \ F$

Come nella valutazione delle espressioni booleane, la valutazione delle espressioni condizionali è “**pigra**”:

quando si valuta `if E then F else G`:

- viene valutata E;
- se il valore di E è true, allora viene valutata F – e G non viene valutata;
- se il valore di E è false, allora viene valutata G – e F non viene valutata.

Esempi

```
# 4 + (if 1 < 0 then 3 * 8 else 5 / 2);;  
- : int = 6
```

```
4 + (if 1 < 0 then 3 * 8 else 5 / 2)  
  ==> 4 + (if false then 3 * 8 else 5 / 2)  
  ==> 4 + (5 / 2)  
  ==> 4 + 2  
  ==> 6
```

```
(* sign : int -> int *)  
(* sign n = 0, 1 o -1 a seconda del "segno" di n *)  
let sign n =  
  if n > 0 then 1  
  else if n = 0 then 0  
       else -1
```

Indentazione: sempre gli else sotto i rispettivi if.

La parte else non manca mai: non ci sono problemi di ambiguità

Regole di calcolo per la valutazione di espressioni

MODELLO DI CALCOLO:
CALCOLARE = RIDURRE

```
# let square x = x * x;;  
val square : int -> int = <fun>  
  square (3*2) ==> square 6 ==> 6 * 6 ==> 36  
  square (3*2) ==> (3*2) * (3*2) ==> 6 * (3*2)  
  ==> 6*6 ==> 36
```

Regole di calcolo:

- **CALL BY VALUE:** calcolare il valore dell'argomento prima di applicare una funzione
- **CALL BY NAME:** applicare la funzione prima di aver calcolato il valore dell'argomento

Regola di calcolo di ML: call by value

Eccezioni: espressioni condizionali, operatori booleani

Necessità di espressioni “lazy”

```
let rec fact n = if n=0 then 1
                  else n * fact(n-1)
```

Call by value:

```
fact 1 ==> if 1 <= 0 then 1 else 1 * fact(1-1)
          ==> if false then 1 else 1 * fact 0
          ==> if false then 1
                else 1 * (if 0<=0 then 1 else 0 * fact(0-1))
          ==> if false then 1
                else 1 * (if true then 1 else 0 * fact(-1))
          ==> if false then 1
                else 1 * (if true then 1
                          else 0 * (if -1<=0 then 1
                                    else fact(-1-1)))
          ==> if false then 1
                else 1 * (if true then 1
                          else 0 * (if true then 1
                                    else fact(-2)))
          .....
          .....
```

Necessità di espressioni “lazy” (II)

Se la valutazione di espressioni condizionali non fosse “pigra”:

```
(* cond : bool * 'a * 'a -> 'a *)  
let cond(c,e1,e2) =  
    if c then e1 else e2
```

L'applicazione di funzioni definite da programma è regolata dalla “call by value”

```
let rec fact n =  
    cond(n=0, 1, n*fact(n-1))
```

```
# fact 1;;
```

Stack overflow during evaluation (looping recursion?).

(E,F)

```
# let a = (3>4, "pippo");;  
val a : bool * string = (false, "pippo")  
# let b = (a, 5.1);;  
val b : (bool * string) * float = ((false, "pippo"), 5.1)
```

$t_1 \times t_2$ è il tipo delle coppie ordinate il cui primo elemento è di tipo t_1 ed il secondo di tipo t_2 (il prodotto cartesiano di t_1 e t_2).

Attenzione: $\times$ è un **costruttore di tipo** (un'operazione su tipi)

```
let c = (double, 6);;
```

(E,F)

```
# let a = (3>4, "pippo");;  
val a : bool * string = (false, "pippo")  
# let b = (a, 5.1);;  
val b : (bool * string) * float = ((false, "pippo"), 5.1)
```

$t_1 \times t_2$ è il tipo delle coppie ordinate il cui primo elemento è di tipo t_1 ed il secondo di tipo t_2 (il prodotto cartesiano di t_1 e t_2).

Attenzione: $\times$ è un **costruttore di tipo** (un'operazione su tipi)

```
let c = (double, 6);;  
val c : (int -> int) * int = (<fun>, 6)
```

Selettori del tipo coppia: **fst**: $\alpha \times \beta \rightarrow \alpha$
snd: $\alpha \times \beta \rightarrow \beta$

```
# snd (fst b);;
```

(E,F)

```
# let a = (3>4, "pippo");;  
val a : bool * string = (false, "pippo")  
# let b = (a, 5.1);;  
val b : (bool * string) * float = ((false, "pippo"), 5.1)
```

$t_1 \times t_2$ è il tipo delle coppie ordinate il cui primo elemento è di tipo t_1 ed il secondo di tipo t_2 (il prodotto cartesiano di t_1 e t_2).

Attenzione: $\times$ è un **costruttore di tipo** (un'operazione su tipi)

```
let c = (double, 6);;  
val c : (int -> int) * int = (<fun>, 6)
```

Selettori del tipo coppia: **fst**: $\alpha \times \beta \rightarrow \alpha$
snd: $\alpha \times \beta \rightarrow \beta$

```
# snd (fst b);;  
- : string = "pippo"
```

Triple, quadruple ... tuple

```
# (true,5*4,"venti");;  
- : bool * int * string = (true, 20, "venti")  
# (3<5,10.3,'K',int_of_string "50");;  
- : bool * float * char * int = (true, 10.3, 'K', 50)  
# (true,("pippo",98),4.0);;  
- : bool * (string * int) * float = (true,("pippo",98),4.)  
# (3<4,("pippo",90+8)) = (true, "pippo", 98);;
```

Triple, quadruple ... tuple

```
# (true, 5*4, "venti");;
- : bool * int * string = (true, 20, "venti")
# (3<5, 10.3, 'K', int_of_string "50");;
- : bool * float * char * int = (true, 10.3, 'K', 50)
# (true, ("pippo", 98), 4.0);;
- : bool * (string * int) * float = (true, ("pippo", 98), 4.)
# (3<4, ("pippo", 90+8)) = (true, "pippo", 98);;
```

Characters 23-42:

```
(3<4, ("pippo", 90+8)) = (true, "pippo", 98);;
                        ^^^^^^^^^^^^^^^^^^^^^
```

Error: This expression has type `bool * string * int`
but an expression was expected of type
`bool * (string * int)`

Il prodotto cartesiano non è associativo: `bool * (string * int) ≠ bool * string * int`

Costruttori e Selettori di un tipo di dati

Ogni tipo di dati è caratterizzato da un insieme di

- un insieme di **costruttori** (costanti + operazioni che “costruiscono” valori di quel tipo)
- un insieme di **selettori** (operazioni che “selezionano” componenti da un valore del tipo)

I tipi semplici (int, float, bool, string, char, ...) non hanno selettori ma solo costruttori:

i costruttori di un tipo di dati semplici sono tutti i valori del tipo

Costruttori e Selettori del tipo coppia

- **costruttore**: **(,)** (insieme di parentesi e virgola).
Applicato a un'espressione di tipo α e una di tipo β , ne costruisce una di tipo $\alpha \times \beta$.
- **selettori**: **fst, snd**.
Applicate a un'espressione di tipo $\alpha \times \beta$, ne riportano i componenti (di tipo α e β , rispettivamente).

fst e **snd** sono funzioni polimorfe, ma attenzione:

```
# fst (double, 6, 'p');;
```

Costruttori e Selettori del tipo coppia

- **costruttore**: **(,)** (insieme di parentesi e virgola).
Applicato a un'espressione di tipo α e una di tipo β , ne costruisce una di tipo $\alpha \times \beta$.
- **selettori**: **fst, snd**.
Applicate a un'espressione di tipo $\alpha \times \beta$, ne riportano i componenti (di tipo α e β , rispettivamente).

fst e **snd** sono funzioni polimorfe, ma attenzione:

```
# fst (double, 6, 'p');
```

```
Characters 4-18:  fst (double, 6, 'p');
```

```
^^^^^^^^^^^^^^^^
```

```
Error: This expression has type (int -> int) * int * char
      but an expression was expected of type 'a * 'b
```

$$P_i 1: 'a \times 'b \times 'c \neq 'd \rightarrow 'a$$

$$P_i 3: 'a \neq 'b \neq 'c \neq 'd \rightarrow 'c$$

Definizioni ricorsive

$$\begin{aligned}n! &= 1 \times 2 \times \dots \times n-1 \times n \\ &= (n-1)! \times n = n \times (n-1)!\end{aligned}$$

Caso “base”:

$$0! = 1$$

```
(* fact: int -> int *)  
let rec fact n =  
  if n=0 then 1  
  else n * fact (n-1)
```

Il fattoriale è “definito in termini di se stesso”, ma per un caso “più facile”.

rec è una parola chiave:

```
# let fact_errore n =  
  if n=0 then 1  
  else n * fact_errore (n-1) ;;
```

Definizioni ricorsive

$$\begin{aligned} n! &= 1 \times 2 \times \dots \times n-1 \times n \\ &= (n-1)! \times n = n \times (n-1)! \end{aligned}$$

Caso “base”:

$$0! = 1$$

```
(* fact: int -> int *)  
let rec fact n =  
  if n=0 then 1  
  else n * fact (n-1)
```

Il fattoriale è “definito in termini di se stesso”, ma per un caso “più facile”.

rec è una parola chiave:

```
# let fact_errore n =  
  if n=0 then 1  
  else n * fact_errore (n-1) ;;  
Error: Unbound value fact_errore
```

Nei linguaggi funzionali “puri” non esistono costrutti di controllo per la realizzazione di cicli quali (**for**, **while**, **repeat**), ma il principale meccanismo di controllo è la ricorsione.

La ricorsione è una tecnica per risolvere problemi complessi riducendoli a problemi più semplici dello stesso tipo.

Per risolvere un problema ricorsivamente occorre:

- 1 Identificare i casi semplicissimi (casi di base) che possono essere risolti immediatamente
- 2 Dato un generico problema complesso, identificare i problemi più semplici dello stesso tipo la cui soluzione può aiutare a risolvere il problema complesso
- 3 Assumendo di saper risolvere i problemi più semplici (**ipotesi di lavoro**), determinare come operare sulla soluzione dei problemi più semplici per ottenere la soluzione del problema complesso

Problema: valutazione di un'espressione aritmetica rappresentata da una stringa

Obiettivo: scrivere un programma che, data una stringa che rappresenta un'operazione aritmetica semplice tra interi non negativi, ne riporti il valore numerico.

Esempio: il valore riportato per la stringa "34+12" è l'intero 46. Le operazioni consentite sono somma, differenza, prodotto, divisione (intera).

Tipo e specifica dichiarativa della funzione principale

evaluate: string -> int

evaluate s = valore numerico dell'espressione rappresentata dalla stringa s.
Errore se s non rappresenta un'espressione aritmetica semplice

Progetto: riduzione a sottoproblemi

Identificazione di un sottoproblema utile:

split_string : string -> int * char * int

split_string s = (n,op,m)

dove n = primo operando,

op = carattere che rappresenta l'operazione,

m = secondo operando.

Per risolvere il problema split_string:

- cercare il primo carattere non numerico:

primo_non_numerico: string -> int

primo_non_numerico s = posizione del primo carattere non numerico
nella stringa s.

Errore se non esiste

- estrarre una parte di una stringa:

substring : string -> int -> int -> string

substring s j k = sottostringa di s che va dalla posizione j alla posizione k.

Apriamo il manuale di OCaml

Nel Modulo Pervasives

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/Pervasives.html>

- **val int_of_string : string -> int**

Convert the given string to an integer.

```
# int_of_string "123";;  
- : int = 123
```

Nel **Modulo String**

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/String.html>

- **val sub : string -> int -> int -> string**

String.sub s start len returns a fresh string of length len, containing the substring of s that starts at position start and has length len.

```
# String.sub "0123456" 2 3;;  
- : string = "234"
```

- **val length : string -> int**

Return the length (number of characters) of the given string.

```
# String.length "pippo";;  
- : int = 5
```

Casi di errore

```
# int_of_string "123pippo";;
```

Casi di errore

```
# int_of_string "123pippo";;
```

Exception: Failure "int_of_string".

```
# String.sub "0123456" 2 30;;
```

Casi di errore

```
# int_of_string "123pippo";;
```

Exception: Failure "int_of_string".

```
# String.sub "0123456" 2 30;;
```

Exception: Invalid_argument "String.sub".

Sottoproblema primo_non_numerico (I)

primo_non_numerico: string -> int

primo_non_numerico s = posizione del primo carattere non numerico
nella stringa s. Errore se non esiste

Algoritmo (iterativo): iniziando con l'indice i=0, si incrementa i fino a che il
carattere in posizione i è numerico. Quando il carattere in posizione i non è
numerico, riportare il valore di i.

Sottoproblema: determinare se un carattere è numerico.

```
(* numeric : char -> bool *)  
let numeric c =
```

Sottoproblema primo_non_numerico (I)

primo_non_numerico: string -> int

primo_non_numerico s = posizione del primo carattere non numerico
nella stringa s. Errore se non esiste

Algoritmo (iterativo): iniziando con l'indice i=0, si incrementa i fino a che il
carattere in posizione i è numerico. Quando il carattere in posizione i non è
numerico, riportare il valore di i.

Sottoproblema: determinare se un carattere è numerico.

```
(* numeric : char -> bool *)  
let numeric c =  
  c >= '0' && c <= '9'
```

Nel Modulo String:

val get : string -> int -> char

String.get s n returns the character at index n in string s.

You can also write s.[n] instead of String.get s n.

Raise Invalid_argument if n not a valid index in s.

Sottoproblema primo_non_numerico (II)

Corpo del ciclo: **loop: string -> int -> int**

loop s i = posizione del primo carattere non numerico in s,
a partire dalla posizione i.

```
let rec loop s i =  
  if not (numeric s.[i]) then i  
  else loop s (i+1)
```

primo_non_numerico s = loop s 0

```
let primo_non_numerico s = loop s 0
```

(inizializza i a 0 prima di entrare nel ciclo)

Funz. Induktion

$$\text{let rec } f\ a = f\ a$$

Dichiarazioni locali

La funzione **loop** serve soltanto come funzione di supporto per **primo_non_numerico**: inutile definirla globalmente

```
let primo_non_numerico s =  
  let rec loop s i =  
    if not (numeric s.[i]) then i  
    else loop s (i+1)  
  in loop s 0
```

La funzione locale “vede” il parametro *s* della funzione principale, che in **loop** non cambia mai.

```
let primo_non_numerico s =  
  (* loop: int -> int *)  
  let rec loop i =  
    if not (numeric s.[i]) then i  
    else loop (i+1)  
  in loop 0
```

Soluzione del problema “substring”

substring : string -> int -> int -> string

substring s j k = sottostringa di s che va dalla posizione j alla posizione k.

Utilizziamo **String.sub : string -> int -> int -> string**

String.sub s start len returns a fresh string of length len,
containing the substring of s that starts
at position start and has length len.

La lunghezza della sottostringa riportata da **substring s j k** è **(k-j)+1**, quindi:

```
let substring s j k =  
  String.sub s j ((k-j)+1)
```

Soluzione del problema “split_string”

split_string : string -> int * char * int

split_string s = (n,op,m), dove, se **i** è la posizione del primo carattere non numerico di **s**:

- **n** è l'intero rappresentato dalla sottostringa di **s** che va dal primo carattere fino a quello in posizione **i-1**
- **op** è il carattere in posizione **i**
- **m** è l'intero rappresentato dalla sottostringa di **s** che va dal carattere in posizione **i+1** fino alla fine della stringa

Usiamo:

- **int_of_string** : string -> int per la conversione di stringhe in interi
- **String.length**: string -> int: **String.lengths s** = numero di caratteri di **s**

```
let split_string s =  
  let i = primo_non_numerico s  
  in (int_of_string (substring s 0 (i-1)),  
      s.[i],  
      int_of_string (substring s (i+1)  
                          ((String.length s)-1)))
```

Soluzione del problema principale “evaluate”

Occorre considerare diversi casi a seconda del carattere che rappresenta l'operazione (che deve essere una delle 4 operazioni)

```
(* evaluate: string -> int *)  
let evaluate s =  
  let (n,op,m) = split_string s  
  in if op='+' then n+m  
     else if op='-' then n-m  
        else if op='*' then n*m  
           else if op='/' then n/m  
              else ???
```

let (n,op,m) = split_string s: sia **n** il primo elemento della tripla riportata come valore da **split_string s**, **op** il secondo e **m** il terzo.

Soluzione del problema principale “evaluate”

Occorre considerare diversi casi a seconda del carattere che rappresenta l'operazione (che deve essere una delle 4 operazioni)

exception **BadOperation**

```
(* evaluate: string -> int *)  
let evaluate s =  
  let (n,op,m) = split_string s  
  in if op='+' then n+m  
     else if op='-' then n-m  
        else if op='*' then n*m  
           else if op='/' then n/m  
              else raise BadOperation
```

let (n,op,m) = split_string s: sia **n** il primo elemento della tripla riportata come valore da **split_string s**, **op** il secondo e **m** il terzo.

Espressioni “let”

Dichiarazione locale di una variabile

let $x = E$ **in** F

Le **espressioni** hanno sempre un tipo e un valore.

Il tipo e il valore di **let** $x = E$ **in** F sono quelli dell'espressione che si ottiene da F sostituendo ovunque x con E .

x è una **variabile locale**:

- x ha un valore (quello dell'espressione E) soltanto all'interno dell'espressione F .
- quando tutta l'espressione **let** $x = E$ **in** F è stata valutata, x non ha più un valore.

```
# let x = 1+2 in x*8;;
```

```
- : int = 24
```

```
# x;;
```

```
Characters 0-1:
```

```
  x;;
```

```
  ^
```

```
Unbound value x
```

let $x = E$ in F

- viene calcolato il valore v di E ;
- la variabile x viene provvisoriamente legata a v ;
- tenendo conto di questo nuovo legame, viene calcolato il valore di F : questo è il valore dell'intera espressione;
- il legame provvisorio di x viene sciolto: x torna ad avere il valore che aveva prima o nessun valore.

Cosa vi ricorda?

let $x = E$ in F

- viene calcolato il valore v di E ;
- la variabile x viene provvisoriamente legata a v ;
- tenendo conto di questo nuovo legame, viene calcolato il valore di F : questo è il valore dell'intera espressione;
- il legame provvisorio di x viene sciolto: x torna ad avere il valore che aveva prima o nessun valore.

Cosa vi ricorda?

let $x = E$ in $F \iff (\text{function } x \rightarrow F) E$

Forma generale delle dichiarazioni locali

Esempio: **let [rec] f <parametri> = <corpo> in E**

DICHIARAZIONE-LET in ESPRESSIONE

- È un'espressione
- Il suo valore è il valore che ha **ESPRESSIONE** nell'ambiente che si ottiene estendendo l'ambiente attuale mediante **DICHIARAZIONE-LET**
- Per valutare

DICHIARAZIONE-LET in ESPRESSIONE

in un ambiente $\mathcal{A}$:

- 1 viene "valutata" **DICHIARAZIONE-LET** in $\mathcal{A}$ (l'ambiente $\mathcal{A}$ viene esteso)
- 2 viene valutata **ESPRESSIONE** nel nuovo ambiente
- 3 viene ripristinato l'ambiente $\mathcal{A}$

Dichiarazioni locali per evitare di calcolare più volte il valore di una stessa espressione

Esempio: dati tre interi n , m e k , determinare il quoziente e il resto di $n+m$ diviso k

```
(* esempio: int * int * int -> int * int *)  
let esempio (n,m,k) = ((n+m)/k, (n+m) mod k)
```

Il valore di $n+m$ viene calcolato 2 volte

```
let esempio(n,m,k) =  
  let somma = n+m  
  in (somma/k, somma mod k)
```

O, equivalentemente:

```
let esempio(n,m,k) =  
  (function somma -> (somma/k, somma mod k)) (n+m)
```

```
let esempio(n,m,k) =  
  let aux somma = (somma/k, somma mod k)  
  in aux (n+m)
```

Esempio: riduzione di una frazione ai minimi termini

Rappresentiamo una frazione mediante una coppia di interi.

```
(* gcd : int -> int -> int *)
(* gcd a b = massimo comun divisore di a e b *)
(* assumendo che almeno uno tra a e b sia diverso da 0 *)
let rec gcd a b =
  if b=0 then (abs a)
  else gcd b (a mod b)

(* fraction : int * int -> int * int *)
let fraction (n,d) =
  let com = gcd n d
  in  (n/com, d/com)
```

Ha senso rendere gcd locale a fraction?

Esempio: riduzione di una frazione ai minimi termini

Rappresentiamo una frazione mediante una coppia di interi.

```
(* gcd : int -> int -> int *)
(* gcd a b = massimo comun divisore di a e b *)
(* assumendo che almeno uno tra a e b sia diverso da 0 *)
let rec gcd a b =
  if b=0 then (abs a)
  else gcd b (a mod b)

(* fraction : int * int -> int * int *)
let fraction (n,d) =
  let com = gcd n d
  in  (n/com, d/com)
```

Ha senso rendere gcd locale a fraction?

Motivi per dichiarare localmente una funzione:

- non ha significato autonomo
- consente di “risparmiare” parametri

Funzioni in forma currificata

```
(* gcd:
   int -> int -> int *)
let rec gcd a b =
  if b=0 then (abs a)
  else gcd b (a mod b)
```

```
(* gcd2:
   int * int -> int *)
let rec gcd2 (a,b) =
  if b=0 then (abs a)
  else gcd2 (b,a mod b)
```

gcd è la **forma currificata** di **gcd2**: calcola gli stessi valori, ma “consuma un argomento alla volta”

Funzioni in forma currificata

```
(* gcd:  
   int -> int -> int *)  
let rec gcd a b =  
  if b=0 then (abs a)  
  else gcd b (a mod b)
```

```
(* times:  
   int -> int -> int *)  
let times m n = m * n
```

```
(* gcd2:  
   int * int -> int *)  
let rec gcd2 (a,b) =  
  if b=0 then (abs a)  
  else gcd2 (b,a mod b)
```

```
(* mult:  
   int * int -> int *)  
let mult (m,n) = m * n
```

gcd è la **forma currificata** di **gcd2**: calcola gli stessi valori, ma “consuma un argomento alla volta”

times è la **forma currificata** di **mult**: calcola gli stessi valori, ma “consuma un argomento alla volta”

times 5: int -> int è un’**applicazione parziale** di **times**.

$$\text{sum (times 5) 1 10} = \sum_{k=1}^{10} (5 \times k)$$

Forma currificata di una funzione su tuple

In generale, f_c è la forma currificata di f se

$$\begin{aligned} f &: t_1 \times \dots \times t_n \rightarrow t \\ f_c &: t_1 \rightarrow (t_2 \rightarrow \dots \rightarrow (t_n \rightarrow t) \dots) \end{aligned}$$

e per ogni $a_1, \dots, a_n$: $f(a_1, \dots, a_n) = (((f_c a_1) a_2) \dots a_n)$

Le parentesi possono essere omesse

- sia nel tipo di f_c (si associa a destra),
- sia nell'applicazione di f_c (si associa a sinistra).

Espressioni per denotare funzioni

- 1 variabili (**times**)
- 2 astrazioni funzionali (**function x -> function y -> x*y**,
o anche **fun x y -> x*y**)
- 3 espressioni funzionali (**comp double (times 5)**)

Altri esempi

```
# let pair x y = (x,y);;  
val pair : 'a -> 'b -> 'a * 'b = <fun>
```

```
# let lessthan x y = y < x;;  
val lessthan : 'a -> 'a -> bool = <fun>
```

lessthan 0 è un predicato: essere minore di 0

```
# let greaterthan x y = y > x;;  
val greaterthan : 'a -> 'a -> bool = <fun>
```

```
# let equal x y = x=y;;  
val equal : 'a -> 'a -> bool = <fun>
```

equal 8 è un predicato: essere uguale a 8

Operazioni predefinite in Ocaml

Molte operazioni predefinite in Ocaml sono in forma currificata

```
# max;;  
- : 'a -> 'a -> 'a = <fun>
```

Le operazioni infisse predefinite sono in forma currificata:

```
# ( * );;  
- : int -> int -> int = <fun>
```

```
# ( = );;  
- : 'a -> 'a -> bool = <fun>
```

Definizione di operatori infissi

```
# let (@@) f g x = f(g x);;  
val @@ : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>
```

```
# double @@ treble;;  
- : int -> int = <fun>
```

```
# @@;;
```

Characters 0-2:

Syntax error

```
# (@@);;  
- : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>
```

```
(* f: int -> int *)  
let f = (times 2) @@ ((+) 100)
```

Gestione dei casi “eccezionali”

OCaml prevede un tipo di dati particolare, quello delle le **eccezioni**:

exn

Le eccezioni consentono di scrivere programmi che segnalano un errore: cioè di definire funzioni parziali.

Procedura di definizione di una funzione parziale:

se caso “eccezionale” allora ERRORE
altrimenti

Esiste un insieme di **eccezioni predefinite**: **Match_failure**,
Division_by_zero,...

Ma l'insieme dei valori del tipo **exn** può essere esteso, mediante la
dichiarazione di eccezioni:

```
(* dichiarazione di eccezione *)  
exception NegativeNumber
```

I **nomi delle eccezioni** iniziano tutti con **una lettera maiuscola**

Come segnalare un errore

Dopo aver dichiarato un'eccezione, l'eccezione può essere **sollevata**:

```
(* fact: int -> int
   fact n solleva l'eccezione NegativeNumber se n
       e' negativo,
       altrimenti riporta il fattoriale di n *)
let rec fact n =
  if n < 0 then raise NegativeNumber
              (* viene "sollevata" l'eccezione *)
  else if n=0 then 1
       else n * fact (n-1)

# fact 3;;
- : int = 6
# fact (-1);;
Exception: NegativeNumber.
```

Propagazione delle eccezioni

```
# 4 * fact (-1) ;;  
Exception: NegativeNumber.
```

Se durante la valutazione di un'espressione **E** viene sollevata un'eccezione, il calcolo del valore di **E** termina immediatamente (il resto dell'espressione non viene valutato), e viene sollevata l'eccezione.

```
(* loop : 'a -> 'b *)  
let rec loop x = loop x;;  
  
# let f=fact(-1) in loop f;;  
Exception: NegativeNumber.
```

Catturare un'eccezione

Un'eccezione può essere **catturata** per implementare procedure di questo tipo:

```
calcolare il valore di E
se nel calcolo di tale valore si verifica un errore
allora .....
```

```
# try 4 * fact(-1)
  with NegativeNumber -> 0;;
- : int = 0
```

Attenzione:

Se **E** è di tipo **exn**:

- 1 **E** può essere il valore di qualunque funzione
- 2 **E** può essere argomento di qualunque funzione

Le eccezioni sono “eccezioni” alla tipizzazione forte.

Uso di eccezioni diverse per identificare il tipo di errore

```
(* primo_non_numerico: string -> int *)  
let primo_non_numerico s =  
  let rec loop i = if not (numeric s.[i]) then i  
                    else loop (i+1)  
  in loop 0  
  
# primo_non_numerico "1234";;  
Exception: Invalid_argument "index out of bounds".
```

Uso di eccezioni diverse per identificare il tipo di errore

```
(* primo_non_numerico: string -> int *)  
let primo_non_numerico s =  
  let rec loop i = if not (numeric s.[i]) then i  
                    else loop (i+1)  
  in loop 0
```

```
# primo_non_numerico "1234";;
```

```
Exception: Invalid_argument "index out of bounds".
```

```
let primo_non_numerico str =  
  (* aux: int -> int *)  
  let rec aux i =  
    if not (numeric str.[i]) then i  
    else aux (i+1)  
  in try aux 0  
     with Invalid_argument "index out of bounds"  
        -> raise BadOperation
```

Uso di eccezioni diverse (II)

```
(* split_string : string -> int * char * int *)  
let split_string s =  
  let i = primo_non_numerico s  
  in (int_of_string (substring s 0 (i-1)), s.[i],  
      int_of_string(substring s (i+1) ((String.length s)-1)))  
  
# split_string "43+abc";;  
Exception: Failure "int_of_string".
```

Uso di eccezioni diverse (II)

```
(* split_string : string -> int * char * int *)
let split_string s =
  let i = primo_non_numerico s
  in (int_of_string (substring s 0 (i-1)), s.[i],
      int_of_string(substring s (i+1) ((String.length s)-1)))

# split_string "43+abc";;
Exception: Failure "int_of_string".
```

exception **BadInt**

```
(* split_string : string -> int * char * int *)
let split_string s =
  let i = primo_non_numerico s
  in try (int_of_string (substring s 0 (i-1)), s.[i],
          int_of_string (substring s (i+1)
                              ((String.length s)-1)))
      with Failure "int_of_string" -> raise BadInt
```

Uso “sporco” delle eccezioni

(* **conta_digits: string -> int** *)

(* **conta_digits s = numero di caratteri numerici in s** *)

Uso “sporco” delle eccezioni

```
(* conta_digits: string -> int *)  
(* conta_digits s = numero di caratteri numerici in s *)  
let conta_digits s =  
  let max_index = (String.length s) - 1 in  
  (* loop: int -> int *)  
  (* loop i = numero di caratteri numerici in s  
    a partire dalla posizione i *)  
  let rec loop i =  
    if i > max_index then 0  
    else if numeric s.[i] then 1 + loop (i+1)  
    else loop (i+1)  
  in loop 0
```

Uso “sporco” delle eccezioni

```
(* conta_digits: string -> int *)
(* conta_digits s = numero di caratteri numerici in s *)
let conta_digits s =
  let max_index = (String.length s) - 1 in
  (* loop: int -> int *)
  (* loop i = numero di caratteri numerici in s
     a partire dalla posizione i *)
  let rec loop i =
    if i > max_index then 0
    else if numeric s.[i] then 1 + loop (i+1)
    else loop (i+1)
  in loop 0

let conta_digits s =
  let rec loop i =
    try if numeric s.[i] then 1 + loop (i+1)
    else loop (i+1)
    with Invalid_argument "index out of bounds" -> 0
  in loop 0
```

3. IF E THEN F ELSE FALSE

DEVONO AVERE
LO STESSO TIPO

11/03/2025

E	F	IF...		E F
1	1	1	→	
1	0	0		
0	-	0		

4. IF E THEN F ELSE TRUE

E	F	IF...		NOT(E) F
1	1	1		
1	0	0		
0	1	1		
0	0	1		

5. IF E THEN TRUE ELSE F

E	F	IF...		E F
1	1	1		
1	0	1		
0	1	1		
0	0	0		

6. IF E THEN FALSE ELSE F

E	F	IF...		NOT(E) F
1	1	0		
1	0	0		
0	1	1		
0	0	0		

Uso di pattern nelle dichiarazioni di valore

```
(* evaluate: string -> int *)  
let evaluate s =  
  let (n,op,m) = split_string s  
  in if op='+' then n+m  
     else .....
```

PATTERN MATCHING

Quando si scrive una dichiarazione della forma **let x = <ESPRESSIONE>**, la variabile **x** è un (caso particolare di) **pattern**.

La forma generale delle dichiarazioni di valore è:

let <PATTERN> = <ESPRESSIONE>

```
# let (x,y) = (3*8, (10<100 or false));;  
val x : int = 24  
val y : bool = true  
# let (n,op,m) = split_string "23+7";;  
val n : int = 23  
val op : char = '+'  
val m : int = 7
```

Cos'è un pattern?

- Un **pattern** è una espressione costituita mediante **variabili** e **costruttori di tipo**.
- Per i tipi introdotti fin qui, i costruttori sono tutti e solo:
 - i **valori** dei tipi **int**, **float**, **bool**, **char**, **string**, **unit**
 - i **costruttori** di tuple (coppie, triple, ...), cioè parentesi e virgole
- Esempi:
 - **x** è un pattern, in quanto variabile;
 - **"pippo"** è un pattern, in quanto valore, quindi costruttore del tipo **string**;
 - **2.0** è un pattern, in quanto valore, quindi costruttore del tipo **float**;
 - **(x,y)** è un pattern, in quanto espressione ottenuta a partire da variabili e costruttori del tipo coppia;
 - **(0,x)** è un pattern, in quanto espressione ottenuta a partire da variabili, costruttori del tipo **int** e costruttori del tipo coppia;
 - **(x,true,y)** è un pattern, in quanto espressione ottenuta a partire da variabili, costruttori del tipo **bool** e costruttori del tipo tripla;
 - **x+y** NON è un pattern in quanto "+" non è un costruttore;
 - **-n** NON è un pattern in quanto "-" non è un costruttore.

Restrizione sui pattern

In un pattern **non possono però esserci occorrenze multiple di una stessa variabile:**

(x,x) NON è un pattern

(con un'eccezione, la “*variabile muta*”)

Pattern matching: confronto con un pattern

Un valore **V** è **conforme a un pattern P** se è possibile sostituire le variabili in **P** con sottoespressioni di **V** in modo tale da ottenere **V** stesso

Pattern matching: confronto di un'espressione **E** con un pattern **P**:

- il confronto ha successo se il valore **V** di **E** è conforme al pattern **P**
- in caso di successo, viene stabilito come sostituire le variabili del pattern **P** in modo da ottenere **V**

```
# let (n,op,m) = split_string "23+7";;  
val n : int = 23  
val op : char = '+'  
val m : int = 7
```

In una dichiarazione **let <PATTERN> = <ESPRESSIONE>**, se il confronto del valore di **<ESPRESSIONE>** con il **<PATTERN>** ha successo, l'ambiente viene esteso aggiungendo i legami delle variabili che risultano dal pattern matching

Pattern matching: esempi (I)

<i>Pattern</i>	<i>Espressione</i>	<i>Pattern Matching</i>
"pippo"	"pippo"	Successo
"pippo"	"pluto"	Fallimento
"pippo"	0	ERRORE
x	"pippo"	$x \Rightarrow \text{"pippo"}$
x	"pi" ^ "ppo"	$x \Rightarrow \text{"pippo"}$
x	3*8	$x \Rightarrow 24$
("pippo", true)	("pippo", true)	Successo
("pippo", true)	("pippo", 3<0)	Fallimento
("pippo", x)	("pippo", 3<0)	$x \Rightarrow \text{false}$

Pattern matching: esempi (II)

<i>Pattern</i>	<i>Espressione</i>	<i>Pattern Matching</i>
x	(2.0, 3<0)	$x \Rightarrow (2.0, \text{false})$
(n,m)	(2.0, 3<0)	$n \Rightarrow 2.0$ $m \Rightarrow \text{false}$
(n,m,k)	(2.0, 3<0)	ERRORE
(n,m,k)	(2.0, 3*8, 3>0)	$n \Rightarrow 2.0$ $m \Rightarrow 24$ $k \Rightarrow \text{true}$
(n,(m,k))	(2.0, 3*8, 3>0)	ERRORE
(n,(m,k))	(2.0, (3*8, 3>0))	$n \Rightarrow 2.0$ $m \Rightarrow 24$ $k \Rightarrow \text{true}$

Nota : Il pattern matching di un pattern costituito da una sola variabile con qualunque espressione, di qualunque tipo, ha sempre esito positivo.

Uso di pattern nelle dichiarazioni di funzioni

Come nelle dichiarazioni di valore, anche nelle dichiarazioni di funzione i parametri sono pattern:

```
(* fraction : int * int -> int * int *)  
let fraction (n,d) =  
    let com = gcd n d  
    in  (n/com, d/com)
```

L'unico parametro di **fraction** è indicato dal pattern **(n,d)**

L'uso dei pattern permette di evitare l'uso di **selettori**:

```
let fraction x =  
    let n = fst x  
    in let d = snd x  
        in let com = gcd n d  
            in  (n/com, d/com)
```

Uso di pattern nelle espressioni funzionali

Analogamente, nelle espressioni funzionali della forma

function x -> <ESPRESSIONE>

la variabile **x** è un caso particolare di pattern.

Più in generale, un'espressione funzionale ha la forma:

function <PATTERN> -> <ESPRESSIONE>

Ad esempio, possiamo definire **fraction** così:

```
let fraction =  
  function (n,d) -> let com = gcd n d  
                    in  (n/com, d/com)
```

Chiamata di funzioni definite mediante l'uso di pattern

```
(* sort : 'a * 'a -> 'a * 'a *)  
let sort (x,y) = if x<y then (x,y)  
                  else (y,x)
```

```
# sort (3*8,100/2);;
```

```
sort (x,y)
```

y	50
x	24
sort	function(x,y) -> ...
...	...

```
- : int * int = (24, 50)
```

sort	function(x,y) => ...
...	...

Forma generale delle espressioni **function**

$$\begin{array}{l} \mathbf{function} \ P_1 \rightarrow E_1 \\ \quad | \ P_2 \rightarrow E_2 \\ \quad \dots \\ \quad | \ P_n \rightarrow E_n \end{array}$$

- I pattern $P_1, \dots, P_n$ devono essere tutti dello stesso tipo T_1 .
- Le espressioni $E_1, \dots, E_n$ devono essere tutte dello stesso tipo T_2 .
- Il tipo dell'espressione `function ...` è: $T_1 \rightarrow T_2$.

Esempio

```
let rec fact = function
  0 -> 1
  | n -> n * fact(n-1)
```

Si possono anche usare “pattern multipli”:

```
let rec fact = function
  0 | 1 -> 1
  | n -> n * fact(n-1)
```

Valutazione dell'applicazione di funzioni definite mediante espressioni **function** generali

$$\begin{array}{l} \text{let } \mathbf{F} = \text{function } Pattern_1 \rightarrow E_1 \\ \quad | Pattern_2 \rightarrow E_2 \\ \quad \dots \\ \quad | Pattern_n \rightarrow E_n \end{array}$$

Per valutare **F(Expr)** (cioè il valore di una funzione **F** applicata ad una espressione **Expr**):

- ① Viene calcolato il valore **V** dell'argomento **Expr**.
- ② Il valore **V** viene confrontato con $Pattern_1, Pattern_2, \dots$, nell'ordine:
Se il confronto dà sempre esito negativo, la valutazione riporta un errore.
Altrimenti:
 - Sia $Pattern_i$ il primo pattern con cui il confronto di **V** ha successo; si aggiungono provvisoriamente i nuovi legami determinati dal pattern matching.
 - Con questi nuovi legami viene valutato il *corpo* della funzione E_i .
 - Il valore di E_i viene riportato come valore di **F(Expr)**.
 - I legami provvisori vengono sciolti.

La variabile muta : esempi di pattern matching

Per non memorizzare
il valore dell'espressione
in una variabile

Pattern	Espressione	Pattern Matching
<code>_</code>	<code>"pippo"</code>	Successo
<code>("pippo", _)</code>	<code>("pippo", 3 < 0)</code>	Successo
<code>(n, _, m)</code>	<code>(2.0, 3 * 8, 3 > 0)</code>	<code>n = 2.0</code> <code>m = true</code>
<code>(_, _)</code>	<code>(2.0, 3 * 8, 3 > 0)</code>	ERRORE
<code>(_, _)</code>	<code>(2.0, (3 * 8, 3 > 0))</code>	Successo

Esempi

```
(* xor: bool * bool -> bool *)  
let xor = function  
  (true,false) | (false,true) -> true  
  | _ -> false
```

```
let primo_non_numerico str =  
  let rec aux i = ...  
  in try aux 0  
    with _ -> raise BadOperation
```

```
let split_string s =  
  let i = primo_non_numerico s  
  in try ....  
    with _ -> raise BadInt
```

```
let conta_digits s =  
  let rec loop i = try ....  
                    with _ -> 0  
  in loop 0
```

match E **with**

$$\begin{array}{l|l} & P_1 \rightarrow E_1 \\ & P_2 \rightarrow E_2 \\ & \dots \\ & P_k \rightarrow E_k \end{array}$$

- I pattern $P_1, \dots, P_n$ devono essere tutti dello stesso tipo, e dello stesso tipo di E .
- Le espressioni $E_1, \dots, E_n$ devono essere tutte dello stesso tipo T .
- Tipo dell'espressione **match** ...: T .

Esempio

```
let rec fact n =  
  match n with  
    0 -> 1  
  | _ -> n * fact (n-1)
```

Cosa succede con questa definizione?

```
let rec fact n = function  
  0 -> 1  
| n -> n * fact (n-1)
```

Esempio

```
let rec fact n =  
  match n with  
    0 -> 1  
  | _ -> n * fact(n-1)
```

Cosa succede con questa definizione?

```
let rec fact n = function  
  0 -> 1  
  | n -> n * fact(n-1)
```

Characters 50-59:

```
  | n -> n * fact(n-1);;  
                ^^^^^^^^
```

Error: This expression has type `int -> int`
 but an expression was expected of type `int`

“evaluate” con un’espressione match

I diversi casi considerati da evaluate si distinguono a seconda della “forma” del carattere che rappresenta l’operatore

```
let evaluate s =  
  let (n,op,m) = split_string s  
  in match op with  
    '+' -> n+m  
  | '-' -> n-m  
  | '*' -> n*m  
  | '/' -> n/m  
  | _ -> raise BadOperation
```

Valutazione di espressioni **match**

match F with $Pattern_1 \rightarrow E_1$
 | $Pattern_2 \rightarrow E_2$
 ...
 | $Pattern_n \rightarrow E_n$

Per valutare l'espressione:

- 1 Viene valutata l'espressione **F**
- 2 Il valore ottenuto viene confrontato con $Pattern_1$, $Pattern_2$, ..., nell'ordine; se il confronto dà sempre esito negativo, la valutazione riporta un errore (**Match_failure**).

Altrimenti:

- Sia $Pattern_i$ il primo pattern con cui il valore di **F** si confronta positivamente: si aggiungono provvisoriamente i nuovi legami determinati dal pattern matching.
- Con questi nuovi legami viene valutata l'espressione E_i .
- Il valore di E_i viene riportato come valore di **Expr**.
- Vengono sciolti i legami provvisori.

Pattern non esaustivi

```
# let evaluate s = let (n,op,m) = split_string s
                    in match op with
                        '+' -> n+m
                        | '-' -> n-m
                        | '*' -> n*m
                        | '/' -> n/m;;
```

Characters 54-127:

```
.....match op with
    '+' -> n+m
  | '-' -> n-m
  | '*' -> n*m
  | '/' -> n/m..
```

Warning 8: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched: 'a'

```
val evaluate : string -> int = <fun>
```

```
# evaluate "35=22";;
```

Exception: Match_failure ("//toplevel//", 16, 5).

Regole di calcolo per la valutazione di espressioni

MODELLO DI CALCOLO: CALCOLARE = RIDURRE

```
# let square x = x * x;;  
val square : int -> int = <fun>  
  square (3*2) ==> square 6 ==> 6 * 6 ==> 36  
  square (3*2) ==> (3*2) * (3*2) ==> 6 * (3*2)  
                    ==> 6 * 6 ==> 36
```

Regole di calcolo:

- **CALL BY VALUE:** calcolare il valore dell'argomento prima di applicare una funzione
- **CALL BY NAME:** applicare la funzione prima di aver calcolato il valore dell'argomento

Regola di calcolo di ML: call by value

Eccezioni: espressioni condizionali, operatori booleani

LET SQUARE $x = x * x$

FUNCTIONAL

LET SQUARE = FUNCTION $x \rightarrow x * x$

SQUARE (3 * 2)

BY NAME

SQUARE (3 * 2) $\rightarrow$ (FUNCTION $x \rightarrow x * x$) (3 * 2)
 $\rightarrow$ (3 * 2) * (3 * 2) $\rightarrow$ 6 * (3 * 2)
 $\rightarrow$ 6 * 6 $\rightarrow$ 36

BY VALUE

SQUARE (3 * 2) $\rightarrow$ (FUNCTION $x \rightarrow x * x$) (3 * 2)
 $\rightarrow$ (FUNCTION $x \rightarrow x * x$) 6
 $\rightarrow$ 6 * 6 $\rightarrow$ 36

Necessità di espressioni “lazy”

```
let rec fact n = if n<=0 then 1
                  else n * fact(n-1)
```

Call by value:

```
fact 1 ==> if 1 <= 0 then 1 else 1 * fact(1-1)
==> if false then 1 else 1 * fact 0
==> if false then 1
      else 1 * (if 0<=0 then 1 else 0 * fact(0-1))
==> if false then 1
      else 1 * (if true then 1 else 0 * fact(-1))
==> if false then 1
      else 1 * (if true then 1
                  else 0 * (if -1<=0 then 1
                              else fact(-1-1)))
==> if false then 1
      else 1 * (if true then 1
                  else 0 * (if true then 1
                              else fact(-2)))
.....
```

Esercizi proposti – 1

1. Se abbiamo dato le seguenti dichiarazioni:

```
let pi = 3.14159;;
let area x = pi *. x;;
let pi = 0.0;;
let x = "pippo";;
```

qual è la risposta di OCaml quando si immette la dichiarazione `let a = area 3.0` ?

Rappresentare l'evoluzione dell'ambiente nel corso della sessione e determinare in quale parte dell'ambiente la funzione `area` “cerca” il valore di `pi`.

2. Dopo aver immesso le seguenti dichiarazioni:

```
let y = 100;;
let x = 5;;
let h x = y+x;;
let y = 0;;
```

qual è il valore dell'espressione `h 7`? Rappresentare l'evoluzione dell'ambiente nel corso della sessione e determinare in quale parte dell'ambiente la funzione `h` “cerca” il valore di `y`.

3. Qual è il tipo della funzione sotto definita?

```
let punct x = x = '.' or x = ',' or x = ';' ;
```

4. definire funzioni `pi1`, `pi2`, `pi3`, `pi4` che riportino, rispettivamente, il primo, secondo, terzo e quarto elemento di una quadrupla. Qual è il loro tipo? Si possono applicare a una quintupla?

Se dichiariamo:

```
let quadrupla =
  (5,('c',"antonio",(),if 3>4 then 0 else 1),"pippo",true)
```

Qual è il valore dell'espressione `pi3 (pi2 quadrupla)` e di `pi4 (pi2 quadrupla)` ?

5. Riscrivere le espressioni seguenti in espressioni equivalenti che non contengano `if/then/else`:

- 1) `if E then true else false`
- 2) `if E then false else true`
- 3) `if E then F else false`
- 4) `if E then F else true`
- 5) `if E then true else F`
- 6) `if E then false else F`

Osservazione importante: si consiglia di tenere bene a mente questo esercizio quando si risolveranno tutti gli esercizi dei gruppi seguenti e gli esercizi d'esame.

1. Se abbiamo dato le seguenti dichiarazioni:

```
let pi = 3.14159;;  
let area x = pi *. x;;  
let pi = 0.0;;  
let x = "pippo";;
```

qual è la risposta di OCaml quando si immette la dichiarazione `let a = area 3.0 ?`

Rappresentare l'evoluzione dell'ambiente nel corso della sessione e determinare in quale parte dell'ambiente la funzione `area` "cerca" il valore di `pi`.

VAR	VAL
x	"pippo"
pi	0.0
AREA	FUNCTION x → pi * x
pi	3.14159

1. $3.0 * 3.14159$

QUI CERCA
IL VALORE DI PI, QUANDO VIENE DICHIARATA

2. Dopo aver immesso le seguenti dichiarazioni:

```
let y = 100;;  
let x = 5;;  
let h x = y*x;;  
let y = 0;;
```

qual è il valore dell'espressione `h 7`? Rappresentare l'evoluzione dell'ambiente nel corso della sessione e determinare in quale parte dell'ambiente la funzione `h` "cerca" il valore di `y`.

VAR	VAL
y	0
h	FUNCTION x → y * x
x	5
y	100

1. 10?

3. Qual è il tipo della funzione sotto definita?

```
let punct x = x = '.' or x = ',' or x = ';' or x = ':'
```

CHAR → Bool

4. definire funzioni p11, p12, p13, p14 che riportino, rispettivamente, il primo, secondo, terzo e quarto elemento di una quadrupla. Qual è il loro tipo? Si possono applicare a una quintupla? **→ NO**

Se dichiariamo:

```
let quadrupla =
  (5, 'c', "antonio", (), if 3>4 then 0 else 1), "pippo", true)
```

Qual è il valore dell'espressione p13 (p12 quadrupla) e di p14 (p12 quadrupla) ?

$p_{11}: ('a * 'b * 'c * 'd) \rightarrow 'a$

$p_{12}: ('a * 'b * 'c * 'd) \rightarrow 'b$

$p_{13}: ('a * 'b * 'c * 'd) \rightarrow 'c$

$p_{14}: ('a * 'b * 'c * 'd) \rightarrow 'd$

5. Riscrivere le espressioni seguenti in espressioni equivalenti che non contengano if/then/else:

- 1) if E then true else false
- 2) if E then false else true
- 3) if E then F else false
- 4) if E then F else true
- 5) if E then true else F
- 6) if E then false else F

Osservazione importante: si consiglia di tenere bene a mente questo esercizio quando si risolveranno tutti gli esercizi dei gruppi seguenti e gli esercizi d'esame.

1. ()

2. IF 3>4 THEN 0 ELSE 1 **QUINDI 1**

1. IF E THEN TRUE ELSE FALSE

E

2. IF E THEN FALSE ELSE TRUE

NOT (E)

3. IF E THEN F ELSE FALSE

E	F	IF...
0	0	0
0	1	0
1	0	0
1	1	1

E 1 F

4. IF E THEN F ELSE TRUE

E	F	IF...
0	0	1
0	1	1
1	0	0
1	1	1

NOT(E) || F

5. IF E THEN TRUE ELSE F

E	F	IF...
0	0	0
0	1	1
1	0	1
1	1	1

E || F

6. IF E THEN FALSE ELSE F

E	F	IF...
0	0	0
0	1	0
1	0	0
1	1	0

NOT(E) 2 F

Esercizi proposti – 2

INDICAZIONI DI CARATTERE GENERALE da tenere presenti nella soluzione di tutti gli esercizi (di questo gruppo e dei successivi):

- Evitare di fare un *cut and paste* della modalità interattiva di OCaml. Riportate qui quello che scrivete su un file che potrebbe essere caricato da OCaml con la direttiva **#use**. Se volete riportare anche i test che avete fatto, metteteli come commenti.
 - Le funzioni vanno definite prima di essere usate.
 - Per ogni funzione “ausiliaria” – anche quando è definita localmente mediante un’espressione **let** – va data una **specifico dichiarativa** (cosa calcola la funzione, quando è applicata ai suoi argomenti) e il **tipo**.
 - Curare l’indentazione.
 - Evitare di definire funzioni equivalenti a funzioni predefinite in OCaml (cioè, guardate il manuale!).
-

1. Definire una funzione `ultime_cifre: int -> int * int` che riporti il valore intero delle due ultime cifre di un `int`. Ad esempio:

```
ultime_cifre 245 = (4,5)
ultime_cifre 5 = (0,5).
```

Se il numero è negativo, il segno va ignorato. Ad esempio

```
ultime_cifre (-245) = (4,5)
ultime_cifre (-5) = (0,5).
```

La funzione non deve mai sollevare eccezioni, ma riportare sempre una coppia di interi.

Si cerchi di fornire una soluzione semplice, che operi direttamente sul numero, anziché passare per la sua rappresentazione come stringa. Si ricordi che n/m e' il risultato della divisione intera, $n \bmod m$ il modulo, e $\text{abs } n$ il valore assoluto.

2. Una cifra è bella se è 0, 3, 7; un numero è bello se la sua ultima cifra è bella e la penultima (se esiste) non lo è. Quindi in particolare le cifre belle sono numeri belli. Definire un predicato `bello: int -> bool` che determini se un numero è bello. La funzione non deve mai sollevare eccezioni, ma riportare sempre un `bool`.
3. (Esercizio 8 pag 45 del libro di testo) Scrivere una funzione `data: int * string -> bool`, che, applicata a una coppia (d, m) , dove d è un intero e m una stringa, determini se la coppia rappresenta una data corretta, assumendo che l'anno non sia bisestile. Si assume che i mesi siano rappresentati da stringhe con caratteri minuscoli ("gennaio", "febbraio", ...).

La funzione non deve mai sollevare eccezioni, ma riportare sempre un `bool`.

1. Definire una funzione `ultime_cifre: int -> int * int` che riporti il valore intero delle due ultime cifre di un int. Ad esempio:

```
ultime_cifre 245 = (4,5)  
ultime_cifre 5 = (0,5).
```

Se il numero è negativo, il segno va ignorato. Ad esempio

```
ultime_cifre (-245) = (4,5)  
ultime_cifre (-5) = (0,5).
```

La funzione non deve mai sollevare eccezioni, ma riportare sempre una coppia di interi.

Si cerchi di fornire una soluzione semplice, che operi direttamente sul numero, anziché passare per la sua rappresentazione come stringa. Si ricordi che a/n e' il risultato della divisione intera, $a \bmod n$ il modulo, e abs n il valore assoluto.

MOD 10 MI DA L'ULTIMA CIFRA DI UN NUMERO

LET ULTIME_CIFRE x =

LET y = ABS (x MOD 10)

IN LET z = ABS (x/10) MOD 10
IN (z,y);;

LET ULTIME_CIFRE x = (ABS (x/10) MOD 10
ABS (x MOD 10))

245 MOD 10

245 MOD 100 = 45

ABS(245/10) MOD 10 = 4

2. Una cifra è bella se è 0, 3, 7; un numero è bello se la sua ultima cifra è bella e la penultima (se esiste) non lo è. Quindi in particolare le cifre belle sono numeri belli. Definire un predicato `bello: int -> bool` che determini se un numero è bello. La funzione non deve mai sollevare eccezioni, ma riportare sempre un bool.

LET BELLO x = PATTERN MATCHING

MATCH ABS(x MOD 10) WITH

0|3|7 -> (MATCH ABS(x MOD 100)/10 WITH

3|7 -> FALSE

1|0 -> x < 100

1_ -> TRUE

1_ -> FALSE

LET BELLO x = NON SOLO PATTERN MATCHING

MATCH (ABS x) < 10 WITH

TRUE -> BELLO x

1_ -> LET (A,B) : ULTIME_CIFRE x

IN BELLO B && NOT BELLO A

3. (Esercizio 8 pag 45 del libro di testo) Scrivere una funzione `data: int * string -> bool`, che, applicata a una coppia (d, m) , dove d è un intero e m una stringa, determini se la coppia rappresenta una data corretta, assumendo che l'anno non sia bisestile. Si assume che i mesi siano rappresentati da stringhe con caratteri minuscoli ('gennaio', 'febbraio', ...).
La funzione non deve mai sollevare eccezioni, ma riportare sempre un `bool`.

```
LET COM M =  
  MATCH M WITH  
  "APRILE" | "GIUGNO" | "SETTEMBRE" | ... -> 30  
  | "FEBBRAIO" -> 28  
  | "GENNAIO" | "MARZO" | ... -> 31  
  | _ -> 0
```

```
LET DATA (D, M) = 0 <= COM M && D > 0
```

Costrutti imperativi

Problema: stampa degli interi compresi tra n e m

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/Pervasives.html>

Output functions on standard output

val **print_string**: string -> unit *TIPO CHE CONTIENE UN SOLO
ELEMENTO LA PARENTESI VUOTA*

Print a string on standard output.

val **print_int**: int -> unit

Print an integer, in decimal, on standard output.

Il tipo unit

Contiene un solo elemento: ()

Sequenze di comandi

(E1 ; E2 ; ... ; En)

Se **E1, ..., En** sono espressioni, allora **(E1 ; E2 ; ... ; En)** è un'espressione.

- Il **tipo e valore** di **(E1 ; E2 ; ... ; En)** sono il tipo e valore di **En**.
- **Valutazione** di **(E1 ; E2 ; ... ; En)**: le espressioni vengono valutate tutte, da sinistra a destra, ma i valori sono ignorati, tranne quello dell'ultima espressione.

```
# (print_int 3; print_string "*"; print_int 8;  
  print_string " = "; print_int(3*8); print_newline();  
  3*8);;
```

3*8 = 24

- : int = 24

```
# (3*8; print_string "ciao\n"; 10);;
```

Characters 1-4:

```
(3*8; print_string "ciao\n"; 10);;
```

Sequenze di comandi

(E1 ; E2 ; ... ; En)

Se **E1**, ..., **En** sono espressioni, allora **(E1 ; E2 ; ... ; En)** è un'espressione.

- Il **tipo e valore** di **(E1 ; E2 ; ... ; En)** sono il tipo e valore di **En**.
- **Valutazione** di **(E1 ; E2 ; ... ; En)**: le espressioni vengono valutate tutte, da sinistra a destra, ma i valori sono ignorati, tranne quello dell'ultima espressione.

```
# (print_int 3; print_string "*"; print_int 8;  
  print_string " = "; print_int(3*8); print_newline();  
  3*8);;
```

3*8 = 24

- : int = 24

```
# (3*8; print_string "ciao\n"; 10);;
```

Characters 1-4:

```
(3*8; print_string "ciao\n"; 10);;  
^^^
```

Warning 10: this expression should have type unit.

ciao

↳ PERCHÉ CALCOLI 3*8 MA NON UNIT

- : int = 10

Come si implementa un ciclo?

Problema: dato un intero m , stampare i numeri compresi tra 0 e m .

Sottoproblema: stampare gli interi compresi tra m e n .

```
(* ciclo: int -> int -> unit
ciclo n m: stampa gli interi da n a m (estremi inclusi) *)
let rec ciclo n m =
  if n>m then ()
  else (print_int n;      (* sequenza di comandi *)
        print_newline();
        ciclo (n+1) m)
(* stampa: int -> unit
   stampa m: stampa degli interi da 0 a m *)
let stampa = ciclo 0
```

IL RISULTATO VIENE RESTITUITO IMMEDIATAMENTE
SENZA ESeguire ALTRE OPERAZIONI DOPO LA CHIAMA

→ QUANDO CHIAMI STAMPA M IN SARÀ INFO RMATA 0

ciclo è **tail recursive**: al ritorno dalla chiamata ricorsiva non si deve fare nulla. Implementa un'iterazione.

conta_digits di nuovo

conta_digits: string -> int

conta_digits s = numero di caratteri numerici in s

```
let conta_digits s =  
  let rec loop i =  
    try if numeric s.[i] then 1 + loop (i+1)  
      else loop (i+1)  
    with _ -> 0  
  in loop 0
```

loop non implementa un'iterazione

Versione iterativa

conta_digits di nuovo

conta_digits: string -> int

conta_digits s = numero di caratteri numerici in s

```
let conta_digits s =  
  let rec loop i =  
    try if numeric s.[i] then 1 + loop (i+1)  
    else loop (i+1)  
  with _ -> 0  
in loop 0
```

↳ STAI ACCESSENDO A
UN INDICE FUORI DALLA
STRINGA

loop non implementa un'iterazione

Versione iterativa

```
let conta_digits s =  
  let rec loop i result =  
    try if numeric s.[i] then loop (i+1) (1+result)  
    else loop (i+1) result  
  with _ -> result  
in loop 0 0
```

INDICE CORRENTE

CONTATORE

0 → INDICE INIZIALE QUANDO 0 ELEMENTI

↳ PARAMETRO

result rappresenta il “risultato parziale” (o “accumulatore”), inizializzato a 0

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/Pervasives.html>

Input functions on standard input

val **read_line**: unit -> string

Flush standard output, then read characters from standard input until a newline character is encountered. Return the string of all characters read, without the newline character at the end.

val **read_int**: unit -> int

Flush standard output, then read one line from standard input and convert it to an integer. Raise Failure "int_of_string" if the line read is not a valid representation of an integer.

Ciclo di lettura da tastiera

Problema: leggere una sequenza di interi terminata da un punto e riportarne la somma

```
(* somma: unit -> int *)  
let rec somma () =  
  let s = read_line ()  
  in if s="." then 0  
     else (int_of_string s) + somma ()
```

Versione iterativa

Ciclo di lettura da tastiera

Problema: leggere una sequenza di interi terminata da un punto e riportarne la somma

```
(* somma: unit -> int *)  
let rec somma () =  
  let s = read_line ()  
  in if s="." then 0  
  else (int_of_string s) + somma ()
```

↳ DATO CHE DEVO FARE QUALCOSA ALTRO
CON LA CHIAMATA RICORSIVA, IN QUESTO CASO UNA
SOMMA, NON È NECESSARIA

Versione iterativa

```
let somma () =  
  (* loop: int -> int, funzione ausiliaria che implementa  
    il ciclo*)  
  let rec loop result = (* result: accumulatore *)  
    let s = read_line ()  
    in if s="." then result  
    else loop ((int_of_string s) + result)  
  in loop 0 (* result inizializzato a 0 *)
```

Specifica dichiarativa delle funzioni ausiliarie

```
(* loop: int -> int *)  
let rec loop result = (* result: accumulatore *)  
  let s = read_line ()  
  in if s="." then result  
     else loop ((int_of_string s) + result)
```

Che cosa calcola **loop** in generale?

loop n = ???

Specifica dichiarativa delle funzioni ausiliarie

```
(* loop: int -> int *)  
let rec loop result = (* result: accumulatore *)  
  let s = read_line ()  
  in if s="." then result  
     else loop ((int_of_string s) + result)
```

Che cosa calcola **loop** in generale?

loop n = n + somma degli interi letti da tastiera

Uso “sporco” delle eccezioni

```
let somma () =  
  let rec loop result =  
    try let n = int_of_string (read_line())  
        in loop (n+result)  
    with _ -> result  
  in loop 0
```

Problema: determinare numero e somma degli interi letti

```
(* numero_somma: unit -> int * int
   riporta numero e somma degli interi letti *)
let rec numero_somma () =
  let s = read_line() in
  if s="." (* terminato? *)
  then (0,0) (* nessun numero letto, somma 0 *)
  else (* s rappresenta un int, leggi gli altri numeri *)
    let (tot,somma) = numero_somma()
    in (tot+1,somma+(int_of_string s))

(* oppure *)
let rec numero_somma () =
  try let n = int_of_string(read_line())
      in let (tot,somma) = numero_somma()
      in (tot+1,somma+n)
  with _ -> (0,0)
```

Versione iterativa di numero_somma

```
let rec numero_somma () =  
  try let n = int_of_string(read_line())  
      in let (tot,somma) = numero_somma()  
      in (tot+1,somma+n)  
  with _ -> (0,0)
```

→ SI PUO' UTILIZZARE
SOLAMENTE IN
MODALITA' ITERATIVA

```
let numero_somma_it () =  
  (* aux: int -> int -> int * int *)  
  let rec aux tot somma = (* due "accumulatori" *)  
    try  
      aux (tot+1) (somma + (int_of_string(read_line())))  
    with _ -> (tot,somma)  
  in aux 0 0
```

↳ AVVIO CON TOT=0 E SOMMA=0

NON VIENE
PASSATA UNA
STRINGA
PARAMETRO
↓
I "CARATTERI"
VENGONO LETTI
DALL'INPUT DELL'
UTENTE FINO A
CHE NON VIENE
SOLLEVATA UN ECCEZIONE
PER AREA INERITO
UN CARATTERE

General input/output functions

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/Pervasives.html>

type **out_channel**

stdout: out_channel

open_out: string -> out_channel

output_string: out_channel -> string -> unit

close_out: out_channel -> unit

type **in_channel**

stdin: in_channel

open_in: string -> in_channel

input_line: in_channel -> string

close_in: in_channel -> unit

media: string -> unit

media <nomefile> legge dal file con il nome dato una sequenza di numeri interi e stampa il numero degli interi letti, la loro somma e la loro media

Vai al codice

Ricorsione e iterazione

```
let rec fact = function 0 -> 1
                        | n -> n * fact (n-1)
```

fact non è tail recursive

```
fact 3 ==> 3 * fact 2
      ==> 3 * (2 * fact 1)
      ==> 3 * (2 * (1 * fact 0))
      ==> 3 * (2 * (1 * 1))
      ==> 3 * (2 * 1)
      ==> 3 * 2 ==> 6
```

Ma il prodotto è associativo, quindi: $3 \times (2 \times \text{fact } 1) = (3 \times 2) \times \text{fact } 1$, e:

```
fact 3 = 3 * fact 2
      = (3 * 2) * fact 1
      = (6 * 1) * fact 0
      = 6 * 1 = 6
```

Non c'è bisogno di aspettare il risultato delle chiamate ricorsive, possiamo eseguire subito il calcolo 3×2 e conservarlo in un “accumulatore” (o risultato parziale) $\implies$ **algoritmo iterativo**

Algoritmo iterativo per il calcolo del fattoriale

```
fact(n) =  
  f <- 1;  
  while (n > 0)  
    do f <- f*n;  
    n <- n-1  
  done;  
  return f
```

Nei linguaggi funzionali non esiste l'assegnazione.

Il ciclo viene implementato mediante un costrutto ricorsivo:

- utilizziamo una funzione ausiliaria che ha un parametro in più (“accumulatore”): i suoi argomenti sono le variabili che vengono “modificate” nel ciclo;
- l'operazione principale richiama quella ausiliaria “inizializzando” l'accumulatore

Implementazione del ciclo mediante una funzione ausiliaria

- Parametri della funzione ausiliaria: le variabili di ciclo (**n** e **f**);
- Corpo della funzione ausiliaria:
 - if** (condizione di uscita dal ciclo)
 - then** (valore da riportare in uscita dal ciclo)
 - else** <chiamata ricorsiva> su
(argomenti modificati come nel ciclo stesso)
- La funzione ausiliaria viene richiamata da quella principale con argomenti uguali ai valori con cui sono inizializzate le variabili di ciclo

Implementazione del ciclo mediante una funzione ausiliaria

- Parametri della funzione ausiliaria: le variabili di ciclo (**n** e **f**);
- Corpo della funzione ausiliaria:
 - if** **n=0** (**condizione di uscita dal ciclo**)
 - then** **f** (**valore da riportare in uscita dal ciclo**)
 - else** **<chiamata ricorsiva>** su **f*n** e **n-1**
(**argomenti modificati come nel ciclo stesso**)
- La funzione ausiliaria viene richiamata da quella principale con argomenti uguali ai valori con cui sono inizializzate le variabili di ciclo (**n** stesso e **f=1**).

```
(* fact' : int -> int *)  
let rec fact' n =  
  (* aux: int -> int -> int  
    il primo argomento e' il "risultato parziale" *)  
  let rec aux f = function  
    0 -> f      (* il "ciclo" termina *)  
  | n -> aux (f*n) (n-1) (* iterazione successiva *)  
  in aux 1 n (* inizializzazione *)
```

SE FINE IL CICLO
TERMINA

NUMERO
CORRENTE

Specifica della funzione ausiliaria

Cosa calcola **aux**? (Redichiariamola a top level)

```
# fact' 5;;  
- : int = 120  
# aux 6 5;;  
- : int = 720
```

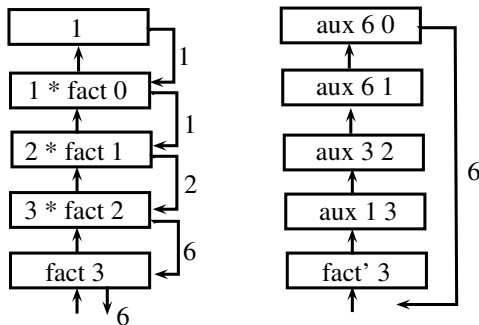
aux m n moltiplica per **m** il fattoriale di **n**

Il processo per il calcolo di fact' è iterativo

```
fact' 3 = aux 1 3  
        = aux 3 2  
        = aux 6 1  
        = aux 6 0 = 6
```

Il processo è “lineare”: dopo aver raccolto il risultato della chiamata ricorsiva, non si deve fare nulla.

Processi ricorsivi e processi iterativi



Processi ricorsivi e iterativi (II)

Un **processo ricorsivo**:

- 1 Esegue calcoli al ritorno dalla ricorsione
- 2 Usa **spazio** proporzionale al numero di chiamate ricorsive

In un **processo iterativo**:

- 1 I calcoli vengono tutti eseguiti prima della chiamata ricorsiva e il risultato parziale viene conservato in un **accumulatore**
- 2 Dopo aver raccolto il risultato della chiamata ricorsiva non si deve fare nulla
- 3 L'ultima chiamata può riportare il suo risultato direttamente alla prima
- 4 Si usa **spazio** costante

Quando un problema P1 viene convertito in un altro P2, in modo che la soluzione di P2 è identica alla soluzione di P1 (non servono altri calcoli), allora

P1 è stato ridotto a P2

P2 è una **riduzione** di P1

Quando una funzione ricorsiva è definita in modo tale che tutte le chiamate ricorsive sono riduzioni, allora la funzione è

RICORSIVA DI CODA (TAIL RECURSIVE)

Molti compilatori riconoscono la ricorsione di coda

Esercizi proposti – 3

1. Rappresentiamo le ore della giornata mediante coppie (h,m) : `int * int`, dove h è compreso tra 0 e 23, inclusi (le ore) e m è compreso tra 0 e 59, inclusi (i minuti). Scrivere un programma con una funzione

```
somma_ore: (int * int) -> (int * int) -> int * int,
```

che calcoli la somma di due ore così rappresentate.

Ad esempio:

```
somma_ore (3,15) (4,20) = (7,35)
```

```
somma_ore (3,45) (4,20) = (8,5)
```

```
somma_ore (23,45) (0,20) = (0,5)
```

Se uno dei due argomenti non è la rappresentazione corretta di un'ora, la funzione solleverà un'eccezione.

Ricordarsi, anche in questo esercizio, le funzioni predefinite di OCaml per calcolare il modulo e la divisione intera.

2. Definire funzioni che risolvano i problemi seguenti (le funzioni avranno il nome e il tipo indicato all'inizio):

- (a) (`read_max: unit -> int`) Leggere da tastiera una sequenza di numeri interi (anche negativi), separati da Enter e terminata dalla stringa vuota (o da una qualsiasi stringa che non rappresenti un intero), e calcolarne il massimo. Se non viene immesso alcun numero, la funzione solleverà un'eccezione.

(Tenere presente la funzione predefinita `max: 'a -> 'a -> 'a`).

- (b) (`read_max_min: unit -> int * int`) Leggere da tastiera una sequenza di numeri interi (anche negativi), separati da Enter e terminata dalla stringa vuota (o da una qualsiasi stringa che non rappresenti un intero), e calcolarne il massimo e il minimo. Se non viene immesso alcun numero, la funzione solleverà un'eccezione.

(Tenere presente le funzioni predefinite `max: 'a -> 'a -> 'a` e `min: 'a -> 'a -> 'a`).

- (c) (`tutti_minori: int -> bool`) Dato un intero n , leggere da tastiera una sequenza di interi (anche negativi), separati da Enter e terminata dalla stringa vuota (o da una qualsiasi stringa che non rappresenti un intero), e determinare (riportando `true` o `false`) se i numeri letti sono tutti minori di n . La funzione non deve mai sollevare eccezioni e la sua esecuzione non deve terminare finché non viene immessa la stringa vuota (o non numerica).

- (d) (`occorre: int -> bool`) Dato un intero n , leggere da tastiera una sequenza di interi, separati da Enter e terminata dalla stringa vuota (o da una qualsiasi stringa non numerica), e determinare (riportando `true` o `false`) se n occorre nella sequenza. La funzione non deve mai sollevare eccezioni e la sua esecuzione non deve terminare finché non viene immessa la stringa vuota (o non numerica).

- (e) (`num_di_stringhe: unit -> int`) Leggere da tastiera una sequenza di stringhe non vuote, separate da Enter e terminata dalla stringa

vuota, e riportare la lunghezza della sequenza (il numero di stringhe immesse, esclusa la stringa vuota che termina la sequenza). La funzione non deve mai sollevare eccezioni.

- (f) (`stringa_max: unit -> string`) Leggere da tastiera una sequenza di stringhe non vuote, separate da Enter e terminata dalla stringa vuota, e riportare la stringa di lunghezza massima. Se non viene immessa nessuna stringa non vuota, la funzione riporterà la stringa vuota.

(Tenere presente la funzione predefinita `String.length: string -> int`).

3. Definire:

- (a) `sumbetween: int -> int -> int`, tale che `sumbetween n m` = somma degli interi compresi tra `n` e `m` (estremi inclusi).
- (b) `sumto: int -> int`, tale che `sumto n` = somma degli interi compresi tra 0 e `n` (incluso), assumendo $n \geq 0$.
- (c) `power: int -> int -> int`, tale che `power n k` = k -esima potenza di `n` (assumendo $n, k \geq 0$).
- (d) `fib: int -> int`, tale che `fib n` = n -esimo numero di Fibonacci (assumendo $n \geq 0$).

La sequenza dei numeri di Fibonacci è così definita:

```
fib 0 = 0,  
fib 1 = 1,  
fib n = fib (n-1) + fib(n-2)
```

- (e) `maxstring: string -> char`, tale che `maxstring s` = massimo carattere in `s` (secondo il codice ASCII).

Ad esempio: `maxstring "antonio" = 't'`. Se l'argomento è la stringa vuota, la funzione solleverà un'eccezione.

Liste

Sequenze finite di elementi di uno stesso tipo:

```
# [1;2;3;4];;  
- : int list = [1; 2; 3; 4]  
# [true; false; 3>0 && 7<=0];;  
- : bool list = [true; false; false]  
# [(1,"pippo");(2,"pluto");(3,"topolino")];;  
- : (int * string) list =  
  [(1, "pippo"); (2, "pluto"); (3, "topolino")]
```

- **list** è un **costruttore di tipi**: se **T** è un tipo, **T list** è il tipo delle liste con elementi di tipo **T**.

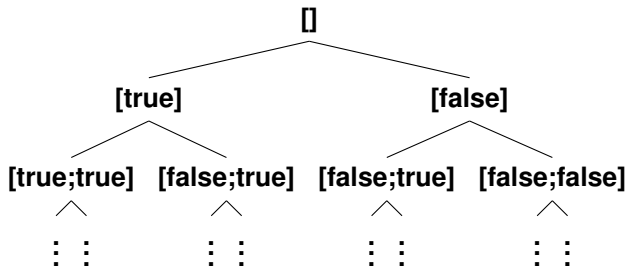
int list
bool list
(int * string) list
'a list

- La **lista vuota** è denotata da **[]**. È un oggetto polimorfo, di tipo **'a list**.
- L'operazione di **inserimento in testa** (**cons**) è denotata da **::**, infisso:
[1;2] = 1::[2] = 1::(2::[])

Definizione induttiva delle liste

- (i) La lista vuota, **[]**, è una α **list**;
- (ii) se **x** è di tipo α e **xs** è una α **list**, allora **(x::xs)** è una α **list**;
- (iii) nient'altro è una α **list**.

La definizione induttiva fornisce un metodo per generare tutte le liste di un dato tipo, per stadi:



Costruttori e selettori delle liste

I **COSTRUTTORI** del tipo **'a list** sono:

```
[] : 'a list  
:: : 'a -> 'a list -> 'a list
```

```
[1;2;3] = 1 :: [2;3] = 1 :: (2 :: [3])  
        = 1 :: (2 :: (3 :: []))
```

I **SELETTORI** del tipo **'a list** sono:

```
List.hd : 'a list -> 'a  
List.tl : 'a list -> 'a list
```

Non sono definiti sulla lista vuota

```
# List.hd [1;2;3;4];;  
- : int = 1  
# List.tl [1;2;3;4];;  
- : int list = [2; 3; 4]  
# List.hd(List.tl [1;2;3;4]);;  
- : int = 2  
# List.hd [];;  
Exception: Failure "hd".  
# List.tl [];;  
Exception: Failure "tl".
```

Definizione ricorsiva di operazioni sulle liste

Le liste si definiscono induttivamente, come i numeri naturali:

	numeri naturali	liste
base	0	[]
operazione per costruire oggetti “più complessi”	+1	::

Sulle liste si possono definire operazioni ricorsivamente:

- si definisce il valore dell'operazione per il caso base **[]**
- assumendo di saper calcolare il valore dell'operazione per una lista **rest** (ipotesi di lavoro), si determina come calcolarne il valore per la generica lista **x::rest** (di cui **rest** è la coda).

Esempio: Lunghezza di una lista

Problema: data una lista **lst** di tipo '**a list**', determinare il numero di elementi di **lst** (la lunghezza della lista).

length: 'a list -> int

Caso base: la lista è vuota. La sua lunghezza allora è 0.

Caso ricorsivo: assumiamo (ipotesi di lavoro) di saper calcolare la lunghezza della coda della lista data: sia n tale lunghezza.

La lunghezza della lista è allora $n + 1$, perché ha un elemento in più rispetto alla sua coda.

Procedimento:

```
length lst: se lst e' vuota, allora riportare 0
            altrimenti (* lst ha un elemento in piu'
                        rispetto alla sua coda *)
                    calcolare la lunghezza n della coda di lst
                    e riportare il valore n+1
```

La definizione è giustificata perché la coda della coda della coda ... della coda di qualsiasi lista è sempre la lista vuota. Quindi prima o poi si arriva al caso base.

Definizione di length

Possiamo usare il test “lista vuota” e il selettore List.tl:

```
(* length: 'a list -> int *)  
let rec length lst =  
  if lst=[] then 0  
  else 1 + (length (List.tl lst))
```

Oppure utilizzando un'espressione **function** generale e il pattern matching:

```
(* length : 'a list -> int *)  
let rec length = function  
  [] -> 0  
  | x::rest -> 1 + length rest
```

Pattern Matching con le liste (I)

```
(* length : 'a list -> int *)  
let rec length = function  
  [] -> 0  
  | x::rest -> 1 + length rest
```

Valutazione di **length [1;2;3;4]**:

- ❶ Il valore dell'argomento è **[1;2;3;4]**
- ❷ **[1;2;3;4]** non è conforme al pattern **[]** (non è la lista vuota), quindi andiamo avanti.
- ❸ **[1;2;3;4]** si può vedere come **1::[2;3;4]**, quindi è conforme al pattern **x::rest**.
Si aggiunge il legame provvisorio di **x** con **1** e di **rest** con **[2;3;4]**.
Con questo legame si calcola il valore di **1 + length rest** (cioè **1 + length [2;3;4]**), che viene riportato come valore di **length [1;2;3;4]**.
I legami provvisori vengono sciolti.

Pattern matching con le liste (II)

In un pattern possono occorrere solo variabili e costruttori:
i costruttori per le liste sono `[]` e `::`

Mediante pattern matching si possono distinguere il caso base e il caso ricorsivo di una definizione ricorsiva sulle liste:

- caso base: `[]` (lista vuota)
- caso ricorsivo: `x::rest` (lista con testa `x` e coda `rest`)

Il pattern matching costituisce un'alternativa all'uso di selettori: i legami determinati dal pattern matching servono a identificare testa e coda di una lista.

Alcuni pattern per le liste:

- `[]`: lista vuota
- `[x]`: lista con un solo elemento, `x`
- `[x;y]`: lista con esattamente due elementi
- `x::rest`: lista con almeno un elemento
- `x::y::rest`: lista con almeno due elementi (`x` è il primo, `y` il secondo, `rest` è la coda della coda).

Alcuni pattern per liste (I)

<i>pattern</i>	<i>espressione</i>		<i>successo e legami</i>
[]	[]		successo
[]	[1]		fallimento
[]	[1;2]		fallimento
[x]	[]		fallimento
[x]	[1]		x=1
[_]	[1;2]		fallimento
x::[]	[]		fallimento
x::[]	[1]	1::[]	x=1
x::[]	[1;2]	1::[2]	fallimento
x::y::[]	[]		fallimento
x::(y::[])	[1]	1::[]	fallimento
x::y::[]	[1;2]	1::2::[]	x=1, y=2
x::y	[]		fallimento
x::y	[1]	1::[]	x=1, y=[]
x::y	[1;2]	1::[2]	x=1, y=[2]
x::y	[1;2;3]	1::[2;3]	x=1, y=[2;3]

Alcuni pattern per liste (II)

<i>pattern</i>	<i>espressione</i>		<i>successo e legami</i>
x::rest	[]		fallimento
x::rest	[1]	1::[]	x=1, rest=[]
x::rest	[1;2]	1::[2]	x=1, rest=[2]
x::rest	[1;2;3]	1::[2;3]	x=1, rest=[2;3]
x::y::rest	[]		fallimento
x::y::rest	[1]	1::[]	fallimento
x::y::rest	[1;2]	1::2::[]	x=1, y=2, rest=[]
x::y::rest	[1;2;3]	1::2::[3]	x=1, y=2, rest=[3]
x::y::rest	[1;2;3;4;5]	1::2::[3;4;5]	x=1, y=2, rest=[3;4;5]



Problema: Date le ultime X estrazioni del superenalotto (sequenze di 6 numeri compresi tra 1 e 90), determinare i 6 numeri che più probabilmente usciranno alla prossima estrazione.

In generale: data una lista contenente liste di interi **estrazioni: int list list**, dove ogni sottolista contiene DIM elementi, compresi tra 1 e HIGHER, determinare i DIM numeri che occorrono nella lista un minor numero di volte.

Problema principale:

super: int list list -> int -> int -> int list

super estrazioni dim higher:

estrazioni è una lista di liste rappresentante le ultime estrazioni,

dim è il numero di interi di ogni estrazione

higher è il massimo numero che può essere estratto

Riporta i **dim** numeri che sono stati estratti meno volte

La schedina del superenalotto: sottoproblemi (I)

- 1) contare, per ogni numero **n** da 1 a **higher**, quante volte **n** occorre in estrazioni.
 - 1a) costruire la lista **[1;2;...;higher]**
Sottoproblema **upto: int -> int -> int list**
upto n m = [n;n+1;...;m]
 - 1b) “appiattare” la lista **estrazioni**, trasformandola in una lista di interi
Sottoproblema **flatten: 'a list list -> 'a list**
 - 1c) “contare” le occorrenze di ciascun elemento di **[1;2;...;higher]** nella lista **flatten estrazioni**:
Sottoproblema **contatutti : 'a list -> 'a list -> ('a * int) list**
contatutti elementi listona = lista di coppie (**ele,n**), dove **ele** è un elemento di **elementi** e **n** è il numero di occorrenze di **ele** in **listona**.
 - 1c-1) contare le occorrenze di un elemento in una lista
Sottoproblema **conta : 'a -> 'a list -> int**
conta n lista = numero di occorrenze di **n** in **lista**

La schedina del superenalotto: sottoproblemi (II)

- 2) ordinare la lista di coppie

contatutti (upto 1 higher) (flatten estrazioni)

secondo valori non decrescenti del secondo elemento

Sottoproblema **sort: ('a * 'b) list -> ('a * 'b) list**

- 3) prendere le prime **dim** coppie della lista ordinata

sort (contatutti (upto 1 higher) (flatten estrazioni))

Sottoproblema **take: int -> 'a list -> 'a list**

take n lista = primi **n** elementi di **lista** (o la lista intera se non ce ne sono abbastanza)

- 4) dalla lista di **dim** coppie ottenuta al punto 3:

take dim (sort (contatutti (upto 1 higher) (flatten estrazioni)))

estrarre la lista con i primi elementi di ciascuna coppia:

Sottoproblema **primi: ('a * 'b) list -> 'a list**

primi [(x1,y1);...;(xn,yn)] = [x1;...;xn]

La schedina del superenalotto: soluzione

[Vai al codice](#)

Merge Sort (ordinamento per fusione)

Algoritmo: Sia C l'insieme (lista) che si vuole ordinare
Si divide l'insieme C in due parti pressappoco uguali.
Ciascuno dei due sottoinsiemi viene ordinato, ricorsivamente.
Si applica l'algoritmo di **fusione** alle liste risultanti dalle chiamate ricorsive

Casi base: lista vuota o di un solo elemento

Caso ricorsivo: per ordinare **lst** di lunghezza > 1 :

- Si suddivide la lista in due parti di dimensioni pressappoco uguali (± 1): siano **xs** e **ys** le due liste ottenute
- Il risultato è la fusione di **(mergesort xs)** e **(mergesort ys)**:
merge (mergesort xs) (mergesort ys)

Vai al codice

Il modulo List

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/List.html>

- **List.hd** : 'a list -> 'a
- **List.tl** : 'a list -> 'a list
- **List.length** : 'a list -> int
- **List.flatten** : 'a list list -> 'a list
- **List.sort** : ('a -> 'a -> int) -> 'a list -> 'a list

GLI DEVI PASSARE
UNA FUNZIONE
DI ORDINAMENTO

Sort a list in increasing order according to a comparison function. The comparison function must return 0 if its arguments compare as equal, a positive integer if the first is greater, and a negative integer if the first is smaller. For example, **compare** is a suitable comparison function.

compare: 'a -> 'a -> int

Vai al codice

Implementazione di procedimenti iterativi: length

Versione ricorsiva

```
(* length : 'a list -> int *)  
let rec length = function  
  [] -> 0  
  | x::rest -> 1 + length rest;;
```

Versione iterativa

```
let len lst =  
  (* aux: int -> 'a list -> int *)  
  (* aux n lst = *)  
  let rec aux result = function  
    [] -> result  
    | _::rest -> aux (result+1) rest  
  in aux 0 lst
```

```
len [1;2;3] = aux 0 [10;20;30] = aux 1 [20;30]  
           = aux 2 [30] = aux 3 [] = 3
```

Specifica dichiarativa di aux?

Implementazione di procedimenti iterativi: length

Versione ricorsiva

```
(* length : 'a list -> int *)  
let rec length = function  
  [] -> 0  
  | x::rest -> 1 + length rest;;
```

Versione iterativa

```
let len lst =  
  (* aux: int -> 'a list -> int *)  
  (* aux n lst = n + lunghezza di lst *)  
  let rec aux result = function  
    [] -> result  
    | _::rest -> aux (result+1) rest  
  in aux 0 lst
```

```
len [1;2;3] = aux 0 [10;20;30] = aux 1 [20;30]  
            = aux 2 [30] = aux 3 [] = 3
```

Specifica dichiarativa di aux?

Calcolo del prodotto degli interi in una lista

Versione ricorsiva:

```
(* prodof : int list -> int *)  
let rec prodof = function  
  [] -> 1  
  | x::xs -> x * prodof xs
```

Versione iterativa

```
let prodof_it lst =  
  (* aux: int -> int list -> int  
    aux n lista = *)  
  let rec aux result = function  
    [] -> result  
    | x::rest -> aux (x*result) rest  
  in aux 1 lst
```

L'accumulatore viene inizializzato con l'elemento neutro per il prodotto

Specifica dichiarativa di aux?

Calcolo del prodotto degli interi in una lista

Versione ricorsiva:

```
(* prodof : int list -> int *)  
let rec prodof = function  
  [] -> 1  
  | x::xs -> x * prodof xs
```

Versione iterativa

```
let prodof_it lst =  
  (* aux: int -> int list -> int  
    aux n lista = n * prodotto degli interi in lista *)  
  let rec aux result = function  
    [] -> result  
    | x::rest -> aux (x*result) rest  
  in aux 1 lst
```

L'accumulatore viene inizializzato con l'elemento neutro per il prodotto

Specifica dichiarativa di aux?

Versione iterativa di upto (I)

```
(* upto : int -> int -> int list *)  
let rec upto m n =  
  if m > n then []  
  else m :: upto (m+1) n
```

Versione iterativa?

```
let upto_it m n =  
  let rec aux result m n =  
    if m > n then result  
    else aux (m::result) (m+1) n  
  in aux [] m n
```

`aux [] 3 5 = aux [3] 4 5 = aux [4;3] 5 5 = ...`

È un **downto** !

Attenzione quando le operazioni che si fanno sull'accumulatore non commutano.

Versione iterativa di upto (II)

Versione iterativa corretta di **upto**: anziché incrementare il limite inferiore, decrementiamo quello superiore:

```
(* upto_it : int -> int -> int list *)
let upto_it m n =
  (* aux: int list -> int -> int -> int list
     aux lista m n = *)
  let rec aux result m n =
    if m > n then result
    else aux (n::result) m (n-1)
  in aux [] m n

upto_it 3 5 = aux [] 3 5
            = if 3>5 then [] else aux [5] 3 4
            = aux [5] 3 4
            = aux [4;5] 3 3
            = aux [3;4;5] 3 2 = [3;4;5]
```

Specifica dichiarativa di aux?

Versione iterativa di upto (II)

Versione iterativa corretta di **upto**: anziché incrementare il limite inferiore, decrementiamo quello superiore:

```
(* upto_it : int -> int -> int list *)
let upto_it m n =
  (* aux: int list -> int -> int -> int list
     aux lista m n = [m;m+1;...;n] @ result *)
  let rec aux result m n =
    if m > n then result
    else aux (n::result) m (n-1)
  in aux [] m n

upto_it 3 5 = aux [] 3 5
            = if 3>5 then [] else aux [5] 3 4
            = aux [5] 3 4
            = aux [4;5] 3 3
            = aux [3;4;5] 3 2 = [3;4;5]
```

Specifica dichiarativa di aux?

Take: versione iterativa?

```
(* take : int -> 'a list -> 'a list *)  
let rec take n = function  
  [] -> []  
  | x::xs -> if n<=0 then [] else  
              x::take (n-1) xs;;
```

→ SE NE A
RICHIEDERE
LA LISTA

PRENDI
N ELEMENTI
DALLA LISTA

Versione iterativa?

```
let rtake n lst =  
  let rec aux result n = function  
    [] -> result  
    | x::rest -> if n<=0 then result else  
                  aux (x::result) (n-1) rest  
  in aux [] n lst  
  
# rtake ([1;2;3;4;5;6], 4);;  
- : int list = [4; 3; 2; 1]
```

Il risultato è rovesciato!

Versione iterativa di take con inserimento in coda

```
let take_it n lst =  
  (* aux : 'a list -> int -> 'a list -> 'a list  
    aux result n lst = *)  
  let rec aux result n = function  
    [] -> result  
    | x::rest -> if n<=0 then result else  
                  aux (result@[x]) (n-1) rest  
  in aux [] n lst
```

Specifica dichiarativa di aux?

Versione iterativa di take con inserimento in coda

```
let take_it n lst =  
  (* aux : 'a list -> int -> 'a list -> 'a list  
    aux result n lst = result @ (take n lst) *)  
  let rec aux result n = function  
    [] -> result  
    | x::rest -> if n<=0 then result else  
                  aux (result@[x]) (n-1) rest  
  in aux [] n lst
```

Specifica dichiarativa di aux?

Ma la concatenazione è un'operazione costosa: conviene inserire gli elementi in testa e rovesciare il risultato alla fine

Rovesciare una lista

Versione ricorsiva

```
(* reverse: 'a list -> 'a list *)  
let rec reverse = function  
  [] -> []  
  | x::xs -> (reverse xs) @ [x];;
```

Versione iterativa

```
(* rev: 'a list -> 'a list *)  
let rev lst =  
  (* revto : 'a list -> 'a list -> 'a list *)  
  (* revto result lst = *)  
  let rec revto result = function  
    [] -> result  
    | x::rest -> revto (x::result) rest  
  in revto [] lst  
  
rev [1;2;3] = revto [] [1;2;3] = revto [1] [2;3]  
           = revto [2;1] [3] = revto [3;2;1] [] = [3;2;1]
```

Specifica dichiarativa di revto?

Rovesciare una lista

Versione ricorsiva

```
(* reverse: 'a list -> 'a list *)
let rec reverse = function
  [] -> []
  | x::xs -> (reverse xs) @ [x];;
```

Versione iterativa

```
(* rev: 'a list -> 'a list *)
let rev lst =
  (* revto : 'a list -> 'a list -> 'a list *)
  (* revto result lst = (reverse lst) @ result *)
  let rec revto result = function
    [] -> result
    | x::rest -> revto (x::result) rest
  in revto [] lst

rev [1;2;3] = revto [] [1;2;3] = revto [1] [2;3]
              = revto [2;1] [3] = revto [3;2;1] [] = [3;2;1]
```

Specifica dichiarativa di revto?

Versione iterativa di take con il reverse

```
let take_it n lst =  
  (* aux : 'a list -> int -> 'a list -> 'a list *)  
  (* aux result n list = *)  
  let rec aux result n = function  
    [] -> List.rev result  
    | x::rest -> if n<=0 then List.rev result else  
                  aux (x::result) (n-1) rest  
  in aux [] n lst
```

Specifica dichiarativa di aux?

Versione iterativa di take con il reverse

```
let take_it n lst =  
  (* aux : 'a list -> int -> 'a list -> 'a list *)  
  (* aux result n list = (rev result) @ (take n lst) *)  
  let rec aux result n = function  
    [] -> List.rev result  
    | x::rest -> if n<=0 then List.rev result else  
                  aux (x::result) (n-1) rest  
  in aux [] n lst
```

Specifica dichiarativa di aux?

Uso delle liste per rappresentare tipi astratti di dati

Il tipo astratto di dati “dizionario”

Il tipo astratto “dizionario” è una collezione di elementi, ciascuno dei quali è costituito da una coppia (**chiave, valore**).

In un dizionario, elementi distinti hanno chiavi distinte.

Un tipo astratto di dati viene caratterizzato dall'**insieme di operazioni** che è possibile fare su tali dati.

Le operazioni sul tipo dizionario sono:

ricerca: dato un dizionario e una chiave, trovare il valore associato alla chiave data

inserimento: dato un dizionario e una coppia (*chiave, valore*), aggiungere tale coppia al dizionario (eventualmente sostituendo l'elemento con la stessa chiave, se esiste)

cancellazione: dato un dizionario e una chiave, cancellare dal dizionario l'elemento con la chiave data, se esiste.

I dizionari si possono implementare in OCaml mediante liste associative.

Liste associative

Una **lista associativa** è un modo di implementare un dizionario, cioè una lista di coppie (associazioni tra chiavi e valori).

Una lista associativa è un **tipo concreto**:

$(\text{'a} \times \text{'b}) \text{ list}$

Le chiavi devono essere di un tipo con uguaglianza.

Ad esempio la lista

```
[ ("pippo", 100); ("pluto", 50); ("topolino", 30) ]
```

è di tipo **(string * int) list** e rappresenta un dizionario con chiavi di tipo **string** e valori associati di tipo **int**.

Problema: dato un dizionario, rappresentato dalla lista associativa **assoc_list**, e una chiave **k**, riportare il valore associato a **k** in **assoc_list**, se esiste, un errore altrimenti.

assoc: 'a -> ('a * 'b) list -> 'b

```
exception NotFound
```

```
(* assoc: 'a -> ('a * 'b) list -> 'b *)
let rec assoc k = function
  [] -> raise NotFound
| (k1,v)::rest -> if k = k1 then v
                  else assoc k rest
```

Nel modulo List: **List.assoc**, solleva l'eccezione predefinita **Not_found** quando la ricerca fallisce

Si consideri la seguente dichiarazione:

```
let rec assoc k lista = function
  [] -> raise NotFound
| (k1,v)::rest -> if k = k1 then v
                  else assoc k rest
```

La dichiarazione non è corretta.

Quale errore viene segnalato da OCaml e perché?

Inserimento

Problema: data una chiave **k**, un valore **v** e una lista associativa **assoc_list**, riportare la lista che si ottiene inserendo la coppia **(k,v)** in **assoc_list** – sostituendo (o sovrascrivendo) l'eventuale elemento già esistente con chiave **k**

inserisci: **'a -> 'b -> ('a * 'b) list -> ('a * 'b) list**

La proprietà fondamentale dell'inserimento è in relazione con la ricerca: si deve avere che

assoc k (inserisci k v assoclist) = v

Sostituire effettivamente un'eventuale coppia **(k,v')** già presente in **assoc_list** è “costoso”: occorre controllare tutta la lista.

Poiché la ricerca riporta il primo valore trovato associato alla chiave della ricerca, è sufficiente inserire la nuova coppia prima delle altre.

```
let inserisci k v assoc_list =  
    (k,v) :: assoc_list
```

Problema: data una chiave **k** e una lista associativa **assoc_list**, riportare la lista che si ottiene cancellando da **assoc_list** tutte le coppie con chiave **k** (potrebbero essercene diverse). Se non c'è nessuna coppia con chiave **k**, riportare **assoc_list** stessa

cancella : 'a -> ('a * 'b) list -> ('a * 'b) list

```
let rec cancella k = function
  [] -> []
| (k',v)::rest ->
    if k=k' then cancella k rest
    else (k',v)::cancella k rest
```

Rappresentazione di tipi astratti di dati (I)

↗ ES. LISTA, COORD, PUA, DIZIONARIO ECC...

Un tipo astratto di dati è costituito da un insieme di oggetti **A** e un insieme di operazioni su tali oggetti.

Per rappresentare un tipo astratto **A**:

1 Determinare un tipo concreto **T** tale che:

- 1 ogni oggetto di **A** abbia almeno un “rappresentante” in **T**;
- 2 elementi distinti di **A** abbiano rappresentanti distinti in **T** (non vengono confusi);

↗ ES. OPERAZIONI SUGLI INSIEMI (ES. AGG. ELEMENTO)

2 Implementare ogni operazione **F** su **A** mediante un'operazione **P** su **T** in modo tale che la rappresentazione “conservi le operazioni”:

↗ APP. CONCETTA
ES. AGG. UN
ELEM.
A UNA LISTA

se $v_1, \dots, v_n$ sono valori di **T** che rappresentano, rispettivamente, gli oggetti $a_1, \dots, a_n$ di **A**, allora

$P(v_1, \dots, v_n)$ è un rappresentante di $F(a_1, \dots, a_n)$

Quindi “operare sui rappresentanti” e poi determinare l'oggetto rappresentato dal valore ottenuto fornisce lo stesso risultato che si ottiene operando sugli oggetti di **A** stessi.

TYPE INSIEME : INT LIST

↓
A

↘
T

Rappresentazione di tipi astratti di dati (II)

Siano $\mathcal{A} = (A, f_1, \dots, f_n)$ e $\mathcal{B} = (B, g_1, \dots, g_n)$ due strutture algebriche dello stesso tipo (per ogni $i = 1 \dots n$, f_i e g_i hanno lo stesso numero di argomenti).

Una **rappresentazione** di $\mathcal{A}$ (tipo astratto) in $\mathcal{B}$ (tipo concreto) è un'applicazione (funzione parziale)

$$\varphi : B \rightarrow A$$

$$(\varphi(b) = a: b \text{ rappresenta } a,$$

quindi ogni $b \in B$ rappresenta al massimo un $a \in A$)

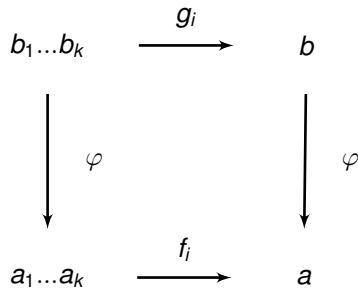
tale che:

- per ogni $a \in A$ esiste $b \in B$ tale che $\varphi(b) = a$ (φ è suriettiva);
- per ogni $i = 1 \dots n$, e per ogni $b_1, \dots, b_n$ di B per cui φ è definita, si ha:

$$\varphi(g_i(b_1, \dots, b_n)) = f_i(\varphi(b_1), \dots, \varphi(b_n))$$

(φ è un omomorfismo)

Rappresentazione di tipi astratti di dati (III)



(Leggere il paragrafo 5.1.2 del libro di testo)

Rappresentazione di insiemi finiti mediante liste (I)

Tipo astratto “insieme finito di elementi dello stesso tipo”, con le operazioni:
 $\in, \cup, \cap, \setminus$

Tipo concreto: liste. Una lista **lst** rappresenta l'insieme che contiene esattamente gli elementi di **lst**.

- Ogni insieme finito di elementi di tipo '**a**' ha almeno un “rappresentante” in '**a list**'. (Quanti?)
- Insiemi distinti hanno rappresentanti distinti in '**a list**': ogni lista rappresenta un unico insieme.

Si devono definire funzioni che rappresentino le operazioni sul tipo astratto:

- mem:** applicata a un elemento x e la rappresentazione di un insieme S , determina se $x \in S$ (è un predicato).
- union:** applicata a due liste che rappresentano insiemi S_1 e S_2 , riporta una rappresentazione di $S_1 \cup S_2$.
- intersect:** applicata a due liste che rappresentano insiemi S_1 e S_2 , riporta una rappresentazione di $S_1 \cap S_2$.
- setdiff:** applicata a due liste che rappresentano insiemi S_1 e S_2 , riporta una rappresentazione di $S_1 \setminus S_2$.

Rappresentazione di insiemi finiti mediante liste (II)

Ad esempio:

<code>[]</code>	rappresenta	$\emptyset$
<code>[1;2;3;4]</code>	rappresenta	$\{1, 2, 3, 4\}$
<code>[4;3;2;1]</code>	rappresenta	$\{1, 2, 3, 4\}$
<code>[3;2;2;2;2;1;2;3;4;2]</code>	rappresenta	$\{1, 2, 3, 4\}$

Per evitare di costruire liste eccessivamente lunghe, si stabilisce che **solo liste senza ripetizioni** rappresentano insiemi: di questo si deve tener conto quando si implementano le operazioni.

Vai al codice

Backtracking

Una delle tecniche più generali per la progettazione di algoritmi

Problemi di ricerca di una soluzione che soddisfi determinate condizioni, e che si possa costruire incrementalmente

Esempio: i candidati sono sequenze $x_1, \dots, x_n$ con $x_i \in S$

Approccio “forza bruta” (enumerazione): generare ad una ad una tutte le sequenze $x_1, \dots, x_n$ e controllare se soddisfano le condizioni

Backtracking: costruire la sequenza aggiungendo un elemento alla volta e controllare passo passo se la sequenza parziale ha possibilità di successo

Ad ogni stadio i : se $x_1, \dots, x_i$ ha possibilità di successo, si sceglie un elemento x_{i+1} tra le diverse alternative; se, con tale scelta, si arriva a una soluzione *sol*, quella è la soluzione.

Altrimenti si sceglie un diverso x_{i+1} .

Se non si trova una soluzione dopo aver esaminato tutte le possibilità: fallimento

Se si verifica che $x_1, \dots, x_i$ non ha possibilità di successo, non si generano tutte le sequenze $x_1, \dots, x_i, \dots$

Riduzione dello spazio di ricerca utilizzando criteri di eliminazione

Somma di sottoinsiemi

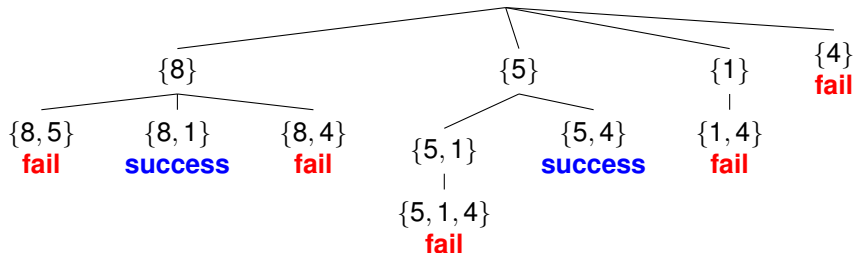
Problema: Dato un insieme S di numeri interi positivi e un intero N , determinare un sottoinsieme Y di S tale che la somma degli elementi di Y sia uguale a N .

Esempio: $S = \{8, 5, 1, 4\}$, $N = 9$.

Aggiungiamo una componente alla volta.

Spazio di ricerca delle soluzioni: albero etichettato da sottoinsiemi di S
Foglie: situazioni non ulteriormente espandibili; alcune di esse possono rappresentare una soluzione (successo) altre no (fallimento).

L'esplorazione dello spazio di ricerca avviene in profondità



Somma di sottoinsiemi: algoritmo

Ad ogni stadio della ricerca:

- una soluzione parziale *Solution*
- un insieme *Altri* di elementi tra cui scegliere

Inizialmente: $Solution = \emptyset$, $Altri = S$

- Se la somma degli elementi di *Solution* è maggiore di N : **fallimento**.
- Se la somma degli elementi di *Solution* è uguale a N : **successo** con *Solution*.

Altrimenti (somma degli elementi di *Solution* minore di N):

- Se *Altri* è vuoto, allora **fallimento**.
- se *Altri* non è vuoto, scegliere un elemento x di *Altri*.

Alternativa:

- mettere x nella soluzione: cercare una soluzione con $\{x\} \cup Solution$ e $Altri - \{x\}$
- non mettere x nella soluzione: cercare una soluzione con *Solution* e $Altri - \{x\}$

Vai al codice

Alcune funzioni del modulo List

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/List.html>

length : 'a list -> int

nth : 'a list -> int -> 'a

assoc : 'a -> ('a * 'b) list -> 'b

rev : 'a list -> 'a list

flatten : 'a list list -> 'a list

mem : 'a -> 'a list -> bool

split : ('a * 'b) list -> 'a list * 'b list

combine : 'a list -> 'b list -> ('a * 'b) list

sort : ('a -> 'a -> int) -> 'a list -> 'a list

merge : ('a -> 'a -> int) -> 'a list -> 'a list -> 'a list

Attraversamento di un labirinto

	0	1	2	3	4
0		argento	mostro		
1	oro	mostro		mostro	
2				mostro	argento
3	mostro	oro			
4			mostro	oro	

Rappresentazione:

- dimensione della matrice
- lista associativa per rappresentare i contenuti

```
let dim = 5
let contents =
  [ ("oro", [(1,0); (3,1); (4,3)]);
    ("argento", [(0,1); (2,4)]);
    ("mostro", [(0,2); (1,1); (1,3); (2,3); (3,0); (4,2)]) ]
```

Vai al codice

Funzioni di ordine superiore

Prendono come argomento o riportano come valore una funzione

Il tipo di una funzione di ordine superiore ha più di una “freccia”

```
let rec sum f lower upper =  
  if lower > upper then 0  
  else f lower + sum f (lower +1) upper
```

Tipo di **sum**: **(int -> int) -> (int -> (int -> int))**

```
(* square : int -> int *)  
let square x = x*x
```

sum square:	int -> int -> int
sum square 3:	int -> int
sum square 3 5:	int

sum square 3 5 = $\sum_{k=3}^5 k^2$

```
(* sumsquare : int -> int -> int *)  
let sumsquare = sum square
```

Funzioni di ordine superiore sulle liste

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/List.html>

Abbiamo già incontrato

List.sort: ('a -> 'a -> int) -> 'a list -> 'a list

List.iter: ('a -> unit) -> 'a list -> unit

Iterators

List.map: ('a -> 'b) -> 'a list -> 'b list

List scanning

List.for_all: ('a -> bool) -> 'a list -> bool

List.exists: ('a -> bool) -> 'a list -> bool

List searching

List.find : ('a -> bool) -> 'a list -> 'a

List.filter: ('a -> bool) -> 'a list -> 'a list

Un esempio di applicazione di map

inits: 'a list -> 'a list list

inits lst = lista con tutti i segmenti iniziali di **lst**

Esempio: `inits [1;2;3;4] = [[1];[1;2];[1;2;3];[1;2;3;4]]`

Casi semplici: **inits []** =
inits [x] =

Un esempio di applicazione di map

inits: 'a list -> 'a list list

inits lst = lista con tutti i segmenti iniziali di **lst**

Esempio: `inits [1;2;3;4] = [[1];[1;2];[1;2;3];[1;2;3;4]]`

Casi semplici: **inits [] = []**
inits [x] = [[x]]

Caso generale: iniziamo a ragionare su un esempio:

```
inits [1;2;3;4]:  
  sia L = inits [2;3;4] = [[2];[2;3];[2;3;4]]  
  si deve applicare l'operazione "inserire 1 in testa a"  
  a tutti gli elementi di L, e aggiungere [1]
```

```
inits (x::rest) =  
  se L = inits rest = [s1;...;sk]  
  ==> [x] :: [x::s1; ....;x::sk]
```

[Vai al codice](#)

Il codice morse

A · —	N — · ·	1 · — — — —
B — · · ·	O — — —	2 · · — — —
C — · — ·	P · — — ·	3 · · · — —
D — · ·	Q — — · —	4 · · · · —
E ·	R · — ·	5 · · · · ·
F · · — ·	S · · ·	6 — · · · ·
G — — ·	T —	7 — — · · ·
H · · · ·	U · · —	8 — — — · ·
I · ·	V · · · —	9 — — — — ·
J · — — —	W · — —	0 — — — — —
K — · —	X — · · —	
L · — · ·	Y — · — —	
M — —	Z — — · ·	

Rappresentazione di insiemi finiti: insieme delle parti

powerset: 'a list -> 'a list list

applicata a una lista L rappresentante un insieme S , riporta una lista con tutti i sottoinsiemi di S .

- Caso base: $L = [] \Rightarrow \text{powerset } L =$

Rappresentazione di insiemi finiti: insieme delle parti



powerset: 'a list -> 'a list list

applicata a una lista L rappresentante un insieme S , riporta una lista con tutti i sottoinsiemi di S .

- Caso base: $L = [] \Rightarrow \text{powerset } L = [[]]$
- Lista non vuota, ragioniamo su un esempio: $L = [1;2;3]$.

Per ipotesi (della ricorsione) si sa calcolare

$$\text{powerset } [2;3] = [[];[2];[3];[2;3]].$$

$$\begin{aligned}\text{powerset } [1;2;3] &= [[];[2];[3];[2;3]] @ [[1];[1;2];[1;3];[1;2;3]] \\ &= [[];[2];[3];[2;3]] @ [1::[]; 1::[2]; 1::[3]; 1::[2;3]]\end{aligned}$$

- Caso generale: $L = x::\text{rest}$
se $[S1;...;Sk]$ sono tutti i sottoinsiemi di **rest**:
$$\text{powerset } L = [S1;...;Sk] @ [x::S1;...;x::Sk]$$

Sottoproblema: operazione di inserimento di un elemento in testa a tutti gli elementi di una lista di liste:

```
(* cons : 'a -> 'a list -> 'a list *)  
let cons x rest = x::rest;;
```

```
List.map (cons x): 'a list list -> 'a list list
```

```
List.map (cons x) [lst1;lst2;...;lstn]  
  = [x::lst1; x::lst2;...; x::lstn]
```

Problema principale:

```
powerset []           ⇒ [[]]  
powerset (x::rest)    ⇒ (powerset rest)  
                      @ (List.map (cons x) (powerset rest))
```

Vai al codice

Prodotto cartesiano di due insiemi

cartprod: 'a list -> 'b list -> ('a * 'b) list

applicata a due insiemi (liste) **setA** e **setB**, riporta la lista di tutte le coppie **(x,y)**, con **x** ∈ **setA** e **y** ∈ **setB**.

Ricorsione sulla prima lista

- Caso base: **cartprod [] setB =**

Prodotto cartesiano di due insiemi

cartprod: 'a list -> 'b list -> ('a * 'b) list

applicata a due insiemi (liste) **setA** e **setB**, riporta la lista di tutte le coppie **(x,y)**, con **x** ∈ **setA** e **y** ∈ **setB**.

Ricorsione sulla prima lista

- Caso base: **cartprod [] setB = []**
- Caso generale: **setA = x::rest**

Per ipotesi (della ricorsione), si sa calcolare **cartprod rest setB**

Si devono aggiungere a questa lista tutte le coppie **(x,y)**, con **y** ∈ **setB**.

Sottoproblema: dato **x** e una lista **lst=[y1;...;yn]**, costruire la lista **[(x,y1);...;(x,yn)]**.

```
let pair x y = (x,y)
    pair: 'a -> 'b -> ('a * 'b)
    pair x:      'b -> ('a * 'b)
    List.map (pair x): 'b list -> ('a * 'b) list
cartprod(x::rest,set) ⇒ (List.map (pair x) set) @ cartprod(rest,set)
```

Vai al codice

Altre utili funzioni del modulo List

<http://caml.inria.fr/pub/docs/manual-ocaml/libref/List.html>

List.rev_map : ('a -> 'b) -> 'a list -> 'b list

List.rev_map f lst = List.rev (List.map f lst)

List.find: ('a -> bool) -> 'a list -> 'a

List.partition: ('a -> bool) -> 'a list -> 'a list * 'a list

List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a

List.fold_left f a [b1; ...; bn] = f (... (f (f a b1) b2) ...) bn.

```
(* sumof : int list -> int *)
```

```
(* sumof [x1; ...; xk] = x1 + ... + xk *)
```

```
let sumof = List.fold_left (+) 0
```

List.fold_right : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a

List.fold_right f [a1; ...; an] b = f a1 (f a2 (... (f an b) ...)).

Definizione di nuovi tipi

Un tipo è un insieme di valori.

Per definire un nuovo tipo occorre specificare:

- 1 un **nome** per il tipo
- 2 come costruire i valori del tipo, cioè quali sono i **costruttori** del tipo.

I tipi enumerati

Sono tipi costituiti da un **insieme finito di valori**. Si possono definire semplicemente **elencando i valori del tipo**.

Ad esempio, il tipo predefinito **bool** è un tipo enumerato.

I valori di un tipo enumerato sono identificati da **costanti**, il modo più semplice di costruire un valore del tipo.

Una costante è dunque un caso particolare di **costruttore**.

Un nuovo tipo viene definito mediante una **dichiarazione di tipo**.

Esempio: il tipo delle quattro direzioni principali

$\{Su, Giu, Destra, Sinistra\}$

Dichiarazione del nuovo tipo **direzione**:

```
type direzione = Su | Giu | Destra | Sinistra;;
```

- **type**: parola chiave che introduce la dichiarazione di un nuovo tipo;
- **direzione**: nome del tipo;
- **Su, Giu, Destra, Sinistra**: valori del nuovo tipo.
- I nomi dei costruttori di tipi definiti dal programmatore **iniziano sempre con una lettera maiuscola**.
- Nella dichiarazione, i valori del tipo sono separati dalla barra verticale.

Una dichiarazione di tipo introduce nuovi valori

Prima della dichiarazione, OCaml non conosce i valori **Su, Giu, Destra, Sinistra**.

```
# Destra;;  
Characters 0-6:  
  Destra;;  
  ^^^^^
```

Unbound constructor Destra

Dopo la dichiarazione:

```
# type direzione = Su | Giu | Destra | Sinistra;;  
type direzione = Su | Giu | Destra | Sinistra  
# Destra;;  
- : direzione = Destra
```

I nuovi valori possono essere utilizzati nella costruzione di altri valori:

```
# [Su; Destra];;  
- : direzione list = [Su; Destra]  
# [(Giu,1); (Sinistra,2)];;  
- : (direzione * int) list = [(Giu, 1); (Sinistra, 2)]
```

Esempio: la rappresentazione delle carte napoletane

Definiamo un tipo per rappresentare i semi e uno per i valori:

```
type seme = Bastoni | Coppe | Denari | Spade;;  
type valore = Asso | Due | Tre | Quattro | Cinque  
             | Sei | Sette | Fante | Cavallo | Re;;
```

Rappresentiamo una carta del mazzo con una coppia di tipo **valore * seme**

```
# let settebello =(Sette,Denari);;  
val settebello : valore * seme = (Sette, Denari)  
  
# let briscola = Spade  
  in [(Asso,Denari);(Due,briscola);(Tre,Bastoni)];;  
  - : (valore * seme) list = [(Asso, Denari);  
                             (Due, Spade); (Tre, Bastoni)]
```

I tipi enumerati sono **tipi con uguaglianza**:

```
# List.mem (Due,Spade) [(Asso,Denari);(Due,Spade);  
                       (Tre,Bastoni)];;  
- : bool = true
```

Le nuove costanti introdotte sono costruttori del tipo

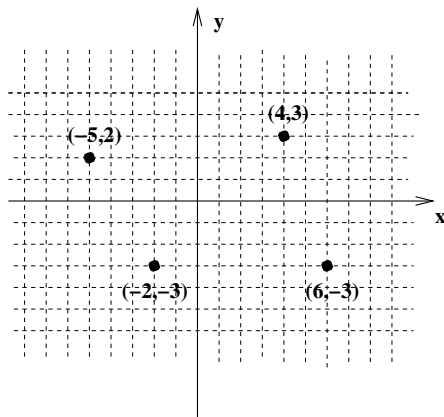
Quindi possono occorrere in un pattern.

E possiamo definire operazioni usando il pattern matching:

```
(* valore : valore -> int *)  
let valore = function  
    Asso -> 1  
  | Due -> 2  
  | Tre -> 3  
  | Quattro -> 4  
  | Cinque -> 5  
  | Sei -> 6  
  | Sette -> 7  
  | Fante -> 8  
  | Cavallo -> 9  
  | Re -> 10;;  
  
# valore Fante;;  
- : int = 8  
# (valore Due) + (valore Asso);;  
- : int = 3
```

Rappresentazione di posizioni e movimenti su un piano

Vogliamo rappresentare la posizione ed il movimento di un oggetto su un piano.



L'oggetto:

- si trova in un punto del piano, identificato da una coppia (x, y) di interi;
- è rivolto in una delle quattro direzioni (Su, Giu, Destra o Sinistra);
- può spostarsi in avanti di n punti, nella direzione verso cui è rivolto;
- può girare in senso orario, modificando la sua posizione di 90° .

Problema: data la posizione (punto del piano (x, y) e una direzione) dell'oggetto e un'azione di spostamento o di cambiamento di direzione, calcolare la nuova posizione dell'oggetto.

Ad esempio:

- se l'oggetto si trova in $(4, 3)$ ed è rivolto verso Destra, quando va avanti di 3 passi si troverà in $(7, 3)$, sempre rivolto verso Destra;
- se l'oggetto si trova in $(-2, -3)$ ed è rivolto verso Giu, quando gira si troverà sempre in $(-2, -3)$, ma rivolto verso Sinistra.

Rappresentazione delle posizioni

Una posizione è identificata da una coppia di coordinate (punto nel piano) e una direzione.

Posizione: punto e direzione

Punto: coppia di interi

Direzione:

```
type direzione = Su | Giu | Destra | Sinistra
```

Possiamo rappresentare le posizioni mediante triple di tipo **int * int * direzione**:

```
type posizione = int * int * direzione
```

Nota bene: non si sta definendo un nuovo tipo, stiamo soltanto dando un altro nome a un tipo già esistente (che OCaml, normalmente, non utilizzerà).

Selettori del tipo posizione

Vai al codice

Rappresentazione delle azioni

Ci sono due tipi di azioni:

- girare di 90° in senso orario
- andare avanti di n passi (con n intero)

Vogliamo definire un tipo di dati **azione** per rappresentare le azioni.

L'azione di girare può essere rappresentata da una costante: **Gira**.

L'azione di andare avanti di n passi costituisce in realtà tutto un insieme di azioni: andare avanti di 0 passi, andare avanti di 1 passo, andare avanti di 2 passi, ... E anche andare avanti di -1 passo (indietro di 1 passo), avanti di -2 passi (indietro di 2), ...

Per rappresentare le azioni serve una funzione: un **costruttore funzionale** che, applicato a un intero n , riporta il valore che rappresenta “andare avanti di n passi”:

Avanti 0, Avanti 1, Avanti 2, Avanti 3, ...
Avanti(-1), Avanti(-2), Avanti(-3),

Dichiarazione del tipo azione

```
type azione = Gira  
            | Avanti of int
```

Questa dichiarazione specifica che l'insieme dei valori di tipo **azione** è costituito da:

- la costante **Gira**;
- tutti i valori della forma **Avanti n**, dove **n** è un intero.

Avanti è un **costruttore funzionale** (si comporta come una funzione):

- è di tipo **int -> azione**;
- quando è applicato a un valore di tipo **int**, restituisce (**costruisce**) un valore di tipo **azione**.

La parola chiave **of** introduce il tipo degli oggetti a cui il costruttore si applica per costruire un oggetto di tipo **azione**.

L'insieme (infinito) dei valori del tipo **azione** è costituito da: **Gira**, **Avanti 0**, **Avanti 1**, **Avanti 2**, **Avanti 3**, ..., **Avanti (-1)**, **Avanti (-2)**, **Avanti (-3)**,

I **costruttori** sono parte della descrizione del tipo, determinano il modo canonico di descrivere i **valori** del tipo.

Pattern matching e selettori

Mediante il pattern matching è possibile definire i **selettori** del tipo:

```
(* int_of_act: azione -> int *)  
let int_of_act = function  
  Avanti n -> n  
  | _ -> failwith "int_of_act"
```

Se definissimo il tipo **posizione** introducendo un nuovo tipo (con il suo costruttore):

```
type posizione = Pos of int * int * direzione
```

Potremmo definirne i selettori come segue:

```
(* xcoord, ycoord : posizione -> int *)  
let xcoord (Pos(x,_,_)) = x  
let ycoord (Pos(_,y,_)) = y  
(* dir : posizione -> direzione *)  
let dir (Pos(_,_,d)) = d
```

ATTENZIONE: in questo caso il tipo **posizione** è diverso da **int * int * direzione**

Qual è l'errore nella definizione seguente?

```
type posizione = Pos of int * int * direzione
```

```
# let punto_di Pos(x,y,_) = (x,y);;
```

Characters 13-16:

```
    let punto_di Pos(x,y,_) = (x,y);;  
                ^^^
```

The constructor Pos expects 3 argument(s),
but is here applied to 0 argument(s)

La soluzione del problema

Funzione principale:

sposta : posizione -> azione -> posizione

sposta pos act: se act ha la forma:

Gira: riporta la posizione che si ottiene girando la direzione di **pos** di 90° in senso orario

Avanti n: riporta la posizione che si ottiene spostando avanti di **n** punti le coordinate di **pos**

Sottoproblema 1: data una direzione, riportare la direzione che si ottiene girando di 90° in senso orario.

Sottoproblema 2: data una posizione **(x,y,d)** e un intero *n*, riportare la posizione **(x',y',d)** che si ottiene andando avanti di *n* passi nella direzione indicata da **d**.

Vai al codice

Unione di tipi

```
type number = Int of int
             | Float of float;;

(* sum : number * number -> number *)
let sum = function
  (Int x, Int y) -> Int (x + y)
| (Int x, Float y) -> Float ((float x) +. y)
| (Float x, Float y) -> Float (x +. y)
| (Float x, Int y) -> Float (x +. float y);;
```

In nuovo tipo rappresenta l'**unione disgiunta** dei tipi che sono gli argomenti dei costruttori

$$A \dot{\cup} B = \{(a, 0) \mid a \in A\} \cup \{(b, 1) \mid b \in B\}$$

I costruttori marcano gli elementi, rendendo riconoscibile la loro provenienza.

Tipi definiti ricorsivamente

Definizione induttiva dei numeri naturali:

- (i) $0 \in \mathbb{N}$
 - (ii) per ogni $n \in \mathbb{N}$, $\text{succ}(n) \in \mathbb{N}$
 - (iii) gli unici elementi di $\mathbb{N}$ sono quelli che si ottengono mediante (i) e (ii).
- Definizione ricorsiva di un nuovo tipo per la rappresentazione dei naturali:

type nat = Zero | Succ of nat

I valori di tipo **nat** sono: **Zero**, **Succ Zero**, **Succ(Succ Zero)**, **Succ(Succ(Succ Zero))**, ...

Sui tipi di dati definiti induttivamente si possono definire operazioni ricorsivamente:

```
(* int_of_nat : nat -> int *)
(* int_of_nat n = valore intero corrispondente a n *)
let rec int_of_nat = function
  Zero -> 0
  | Succ n -> succ(int_of_nat n)
# int_of_nat (Succ (Succ (Succ (Succ Zero))));;
- : int = 4
```

L'operazione somma di numeri naturali

Si può definire ricorsivamente sul primo argomento:

```
(* somma : nat * nat -> nat *)  
let rec somma (n,m) =  
  match n with  
    Zero -> m  
  | Succ k -> Succ(somma(k,m));;  
  
# let k = somma(Succ(Succ Zero), Succ(Succ(Succ Zero)));;  
val k : nat = Succ (Succ (Succ (Succ (Succ Zero))))  
  
# int_of_nat k;;  
- : int = 5
```

Definizione di costruttori di tipi

La definizione di un tipo può essere **parametrica**:

- con variabili di tipo
- con costruttori polimorfi

```
type 'a mylist = Nil | Cons of 'a * 'a mylist
```

Possiamo definire un costruttore di tipi (un tipo polimorfo) per le matrici di interi, matrici di stringhe, matrici di liste di booleani, ...

```
type 'a matrix = Mat of 'a list list
```

matrix è un **costruttore di tipi**

Il costruttore **Mat**: **'a list list -> 'a matrix** è polimorfo

```
Mat [[true;false];[false;true]] : bool matrix  
Mat [[1;2;3];[4;5]] : int matrix
```

Il tipo 'a option: un'alternativa all'uso delle eccezioni

type 'a option = None | Some of 'a

Una funzione parziale di tipo 'a -> 'b può essere implementata da una funzione di tipo 'a -> 'b option.

Utile quando non si vuole propagazione degli errori.

→ DIVISIONE
PER ZERO

```
(* new_assoc : ('a * 'b) list -> 'a -> 'b option *)  
let rec new_assoc lst x =  
  match lst with  
  | [] -> None  
  | (k,v)::rest -> if x=k then Some v  
                    else new_assoc rest x
```

Vai al codice

Espressioni aritmetiche

Consideriamo espressioni costruite a partire da variabili e costanti intere mediante applicazione delle operazioni di somma, sottrazione, prodotto e divisione (intera).

Ad esempio:

$$(3 + (5 \times x)) / (9 - y)$$

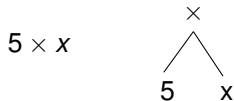
L'insieme delle espressioni aritmetiche si può **definire induttivamente**:

- (i) se n è un numero intero, allora n è un'espressione;
- (ii) se x è una variabile, allora x è un'espressione;
- (iii) se E_1 e E_2 sono espressioni, allora anche $(E_1 + E_2)$, $(E_1 - E_2)$, $(E_1 \times E_2)$ e (E_1 / E_2) sono espressioni;
- (iv) nient'altro è un'espressione.

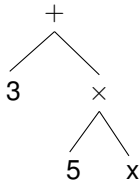
- 1 L'insieme così definito contiene un numero infinito di “oggetti di base”.
- 2 Ci sono quattro “costruttori funzionali” (somma, sottrazione, moltiplicazione, divisione): quattro modi diversi di costruire oggetti complessi a partire da oggetti più semplici.
- 3 I costruttori funzionali si applicano a **due** oggetti più semplici per costruire un oggetto più complesso.

Strutture ad albero

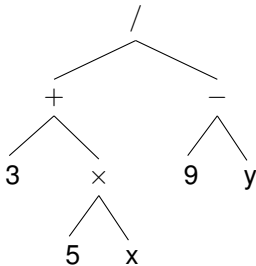
Poiché i costruttori funzionali si applicano a più di un oggetto la struttura di un'espressione aritmetica non è lineare (come quella di una lista), ma è una struttura ad albero.



$(3 + (5 \times x))$



$(3 + (5 \times x)) / (9 - y)$



Rappresentazione delle espressioni in OCaml

Possiamo definire un tipo di dati induttivo, ricalcando la definizione induttiva delle espressioni:

```
type expr =  
  Int of int  
| Var of string  
| Sum of expr * expr  
| Diff of expr * expr  
| Mult of expr * expr  
| Div of expr * expr
```

<i>espressione</i>	<i>rappresentazione</i>
4	Int 4
$4 \times x$	Mult (Int 4, Var "x")
$7 - 3$	Diff(Int 7, Int 3)
$3 + (y \times 2)$	Sum (Int 3, Mult (Var "y", Int 2))

Problema: valutazione di un'espressione in un ambiente

Rappresentazione di ambienti mediante liste associative:

```
type ambiente = (string * int) list
```

[("x", 5) ; ("z", 0) ; ("y", 4)] rappresenta l'ambiente

x	5
z	0
y	4

eval: ambiente -> expr -> int

eval env e = valore dell'espressione **e** nell'ambiente **env**. Errore se qualche variabile in **e** non ha un valore associato in **env**.

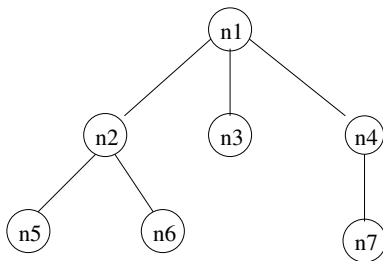
Vai al codice

Un albero è un insieme di oggetti, chiamati **nodi**, su cui è definita una relazione binaria $G(n, m)$ – che leggiamo “ n è **genitore** di m ” – tale che:

- 1 esiste un unico nodo, chiamato **radice**, che non ha genitori;
- 2 ogni nodo diverso dalla radice ha uno ed unico genitore;
- 3 per ogni nodo n diverso dalla radice esiste un **cammino** dalla radice a n (l'albero è *connesso*):
esistono nodi $n_1, \dots, n_k$ ($k \geq 1$) tali che n_1 è la radice dell'albero, $n_k = n$ e, per ogni $i = 1, \dots, k - 1$, n_i è genitore di n_{i+1} .

Se n è il genitore di m , allora m è un **figlio** di n .

Un **albero binario** è un albero in cui ogni nodo ha al massimo due figli.



Un po' di terminologia

cammino: sequenza di nodi $n_1, \dots, n_k, n_{k+1}$ tale che, per $i = 1, \dots, k$, n_i è il genitore di n_{i+1} .

lunghezza del cammino: k (numero degli archi)

antenato e discendente: se esiste un cammino da n a m , allora n è un antenato di m e m un discendente di n

fratelli: nodi che hanno lo stesso genitore

sottoalbero: insieme costituito da un nodo n e tutti i suoi discendenti; n è la radice del sottoalbero

foglia: nodo senza figli

nodo interno: nodo con uno o più figli

profondità di un nodo: lunghezza del cammino dalla radice al nodo

altezza di un nodo: lunghezza del cammino più lungo che va dal nodo a una foglia

altezza dell'albero: altezza della sua radice = profondità massima di un nodo nell'albero

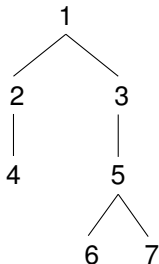
dimensione di un albero: numero dei nodi

Definizione degli alberi binari per induzione strutturale

- 1 Un nodo n è un albero binario (una foglia), con radice n .
- 2 Se T_0 è un albero binario con radice n_0 e n è un nuovo nodo, allora la struttura che si ottiene aggiungendo n come genitore di n_0 è un albero binario con radice n .
- 3 Se T_0 e T_1 sono alberi binari con radici n_0 e n_1 , rispettivamente, e n è un nuovo nodo, allora la struttura che si ottiene aggiungendo n come genitore di n_0 e di n_1 è un albero binario con radice n .
- 4 Nient'altro è un albero binario.

Rappresentazione di alberi binari

```
type 'a tree =  
  Leaf of 'a  
| One of 'a * 'a tree  
| Two of 'a * 'a tree * 'a tree
```



```
Two(1, One(2, Leaf 4),  
    One(3, Two(5, Leaf 6, Leaf 7)))
```

è un **int tree**

Il tipo di un albero binario dipende dal tipo dei nodi:

- **Leaf, One, Two** sono costruttori polimorfi

Leaf: 'a -> 'a tree

One: 'a * 'a tree -> 'a tree

Two: 'a * 'a tree * 'a tree -> 'a tree

- **tree** è un costruttore di tipi

Calcolo della dimensione di un albero binario

Definizione induttiva del tipo $\Rightarrow$ definizione ricorsiva di funzioni sul tipo

```
type 'a tree =  
  Leaf of 'a  
  | One of 'a * 'a tree  
  | Two of 'a * 'a tree * 'a tree  
  
(* size : 'a tree -> int *)  
(* size t = numero di nodi in t *)  
let rec size = function  
  Leaf _ ->  
  | One(_,t) ->  
  
  | Two(_,t1,t2) ->
```

Calcolo della dimensione di un albero binario

Definizione induttiva del tipo $\Rightarrow$ definizione ricorsiva di funzioni sul tipo

```
type 'a tree =  
  Leaf of 'a  
  | One of 'a * 'a tree  
  | Two of 'a * 'a tree * 'a tree  
  
(* size : 'a tree -> int *)  
(* size t = numero di nodi in t *)  
let rec size = function  
  | Leaf _ -> 1  
  | One(_,t) ->  
      (* ipotesi: si sa calcolare size t *)  
  | Two(_,t1,t2) ->
```

Calcolo della dimensione di un albero binario

Definizione induttiva del tipo $\Rightarrow$ definizione ricorsiva di funzioni sul tipo

```
type 'a tree =  
  Leaf of 'a  
  | One of 'a * 'a tree  
  | Two of 'a * 'a tree * 'a tree  
  
(* size : 'a tree -> int *)  
(* size t = numero di nodi in t *)  
let rec size = function  
  Leaf _ -> 1  
  | One(_,t) -> 1 + size t  
    (* ipotesi: si sa calcolare size t *)  
  | Two(_,t1,t2) ->  
    (* ipotesi: si sa calcolare size t1 e size t2 *)
```

Calcolo della dimensione di un albero binario

Definizione induttiva del tipo $\Rightarrow$ definizione ricorsiva di funzioni sul tipo

```
type 'a tree =  
  Leaf of 'a  
  | One of 'a * 'a tree  
  | Two of 'a * 'a tree * 'a tree  
  
(* size : 'a tree -> int *)  
(* size t = numero di nodi in t *)  
let rec size = function  
  Leaf _ -> 1  
  | One(_,t) -> 1 + size t  
    (* ipotesi: si sa calcolare size t *)  
  | Two(_,t1,t2) -> 1 + size t1 + size t2  
    (* ipotesi: si sa calcolare size t1 e size t2 *)
```

Rappresentazione alternativa degli alberi binari

Per rendere le definizioni più compatte, è utile includere l'insieme vuoto tra gli alberi binari: **l'albero "vuoto" (`Empty`)**.

In questo modo, ogni albero diverso da **`Empty`** ha esattamente due sottoalberi: se è una foglia, i sottoalberi sono vuoti, se è un nodo con un solo figlio, uno dei due sottoalberi è vuoto.

Definizione induttiva

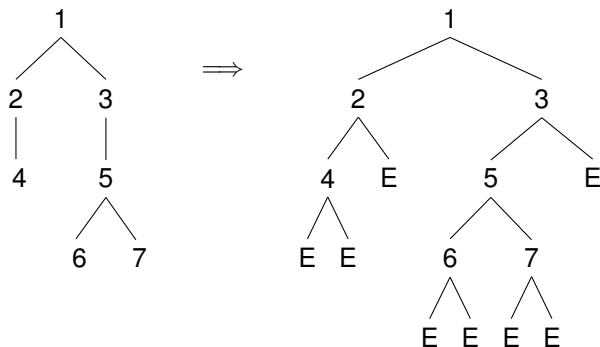
- 1 **`Empty`** è un albero binario
- 2 Se **`t1`** e **`t2`** sono alberi binari e **`n`** è un nuovo nodo, allora **`Tr(n,t1,t2)`** è un albero binario
- 3 Nient'altro è un albero binario

Costruttori: **`Empty`** : 'a tree

`Tr` : 'a * 'a tree * 'a tree -> 'a tree

Dichiarazione di tipo: **`type 'a tree = Empty`**
| **`Tr of 'a * 'a tree * 'a tree`**

Esempio



```
Tr(1, Tr(2, Tr(4, Empty, Empty), Empty),  
    Tr(3, Tr(5, Tr(6, Empty, Empty), Tr(7, Empty, Empty)),  
    Empty))
```

Alcune funzioni di base: predicati e selettori

is_empty: 'a tree -> bool

```
let is_empty = function  
  Empty -> true  
  | _ -> false
```

root: 'a tree -> 'a

```
exception EmptyTree
```

```
let root = function  
  Empty -> raise EmptyTree  
  | Tr(x,_,_) -> x
```

Alcune funzioni di base: predicati e selettori

is_empty: 'a tree -> bool

```
let is_empty = function
  Empty -> true
  | _ -> false
```

root: 'a tree -> 'a

```
exception EmptyTree

let root = function
  Empty -> raise EmptyTree
  | Tr(x,_,_) -> x
```

is_leaf: 'a tree -> bool

```
let is_leaf = function
  Tr(_,Empty,Empty) -> true
  | _ -> false
```

leaf: 'a -> 'a tree (utility)

```
let leaf x =
  Tr(x,Empty,Empty)
```

Alcune funzioni di base: predicati e selettori

is_empty: 'a tree -> bool

```
let is_empty = function
  Empty -> true
  | _ -> false
```

root: 'a tree -> 'a

```
exception EmptyTree
```

```
let root = function
  Empty -> raise EmptyTree
  | Tr(x,_,_) -> x
```

is_leaf: 'a tree -> bool

```
let is_leaf = function
  Tr(_,Empty,Empty) -> true
  | _ -> false
```

leaf: 'a -> 'a tree (utility)

```
let leaf x =
  Tr(x,Empty,Empty)
```

left e right: 'a tree -> 'a tree

```
let left = function
  Empty ->
    raise EmptyTree
  | Tr(_,t,_) -> t
```

```
let right = function
  Empty ->
    raise EmptyTree
  | Tr(_,_,t) -> t
```

size : 'a tree -> int

Usando il pattern matching:

```
let rec size = function
  Empty -> 0
  | Tr(_,t1,t2) -> 1 + size t1 + size t2;;
```

Usando predicati e selettori:

```
let rec size t =
  if is_empty t then 0
  else 1 + size (left t) + size (right t);;
```

Vai al codice

Visita di un albero con risultato parziale

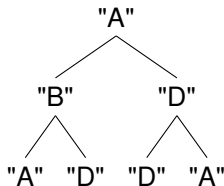
Problema: contare quante volte occorre in un albero ciascuna etichetta.

count : 'a tree -> ('a * int) list

count t = lista di coppie contenente, per ogni etichetta **x** di qualche nodo dell'albero, una (unica) coppia **(x,n)**, dove **n** è il numero di nodi etichettati da **x**.

Algoritmo 1:

- **count Empty** = [] (e questo è facile)
- **count (Tr(x,left,right))**: per ipotesi della ricorsione sappiamo calcolare **count left** e **count right**.



count left=[("B",1);("A",1);("D",1)],
count right=[("D",2);("A",1)]

vanno “fuse” in [("B",1);("A",2);("D",3)],

e poi si deve “aggiungere” la radice, ottenendo
[("B",1);("A",3);("D",3)]

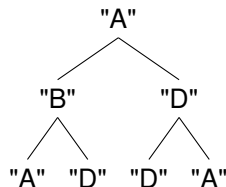
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a **[]**

Visita in preordine:



result=[]

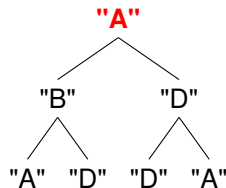
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]
```

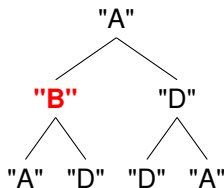
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]
```

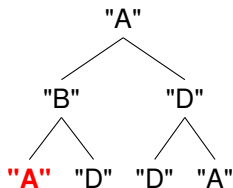
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]
```

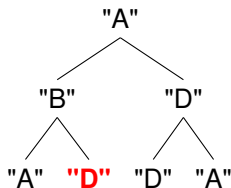
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]  
result=[("A",2);("B",1);("D",1)]
```

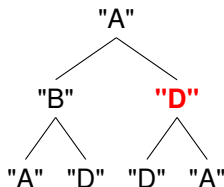
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]  
result=[("A",2);("B",1);("D",1)]  
result=[("A",2);("B",1);("D",2)]
```

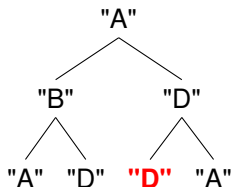
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a **[]**

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]  
result=[("A",2);("B",1);("D",1)]  
result=[("A",2);("B",1);("D",2)]  
result=[("A",2);("B",1);("D",3)]
```

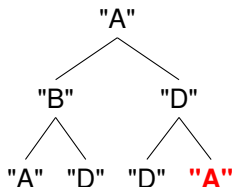
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia **(a,n)**, se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia **(a,1)**.

Risultato parziale **result**, inizializzato a []

Visita in preordine:



```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]  
result=[("A",2);("B",1);("D",1)]  
result=[("A",2);("B",1);("D",2)]  
result=[("A",2);("B",1);("D",3)]  
result=[("A",3);("B",1);("D",2)]
```

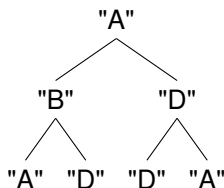
Algoritmo alternativo

Algoritmo 2: visitare l'albero costruendo via via la lista di coppie. Durante la visita si dispone dunque di un risultato parziale: la lista che rappresenta il risultato del procedimento per i nodi già visitati.

Visitare un nodo **a** significa scandire il risultato parziale, aggiungendo 1 al secondo elemento della coppia (**a,n**), se una tale coppia esiste, o altrimenti, se non esiste, aggiungendo al risultato parziale la coppia (**a,1**).

Risultato parziale **result**, inizializzato a []

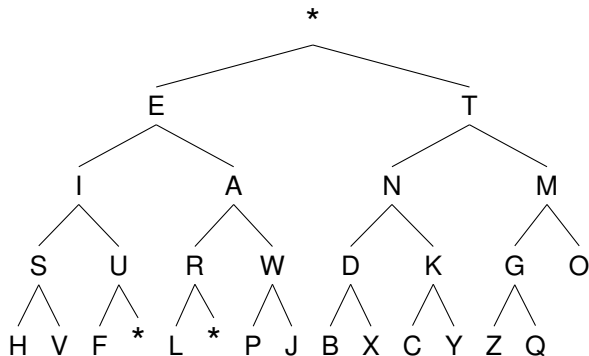
Visita in preordine:



Vai al codice

```
result=[]  
result=[("A",1)]  
result=[("A",1);("B",1)]  
result=[("A",2);("B",1)]  
result=[("A",2);("B",1);("D",1)]  
result=[("A",2);("B",1);("D",2)]  
result=[("A",2);("B",1);("D",3)]  
result=[("A",3);("B",1);("D",2)]
```

L'albero del codice morse (caratteri alfabetici)



Vai al codice

Definizione degli alberi n-ari per induzione strutturale

- 1 L'albero costituito da un unico nodo r senza genitori è un albero, con radice r ;
- 2 Se $t_1, t_2, \dots, t_n$ sono alberi, con radici, rispettivamente, $r_1, r_2, \dots, r_n$, e r è un nuovo nodo, allora la struttura che si ottiene aggiungendo il nodo r come genitore di $r_1, r_2, \dots, r_n$ è un albero;
- 3 Nient'altro è un albero

Rappresentazione

```
type 'a ntree = Tr of 'a * 'a ntree list
```

- Una foglia etichettata da **n** è rappresentata da **Tr(n,[])** (non ha sottoalberi)

```
(* leaf: 'a -> 'a ntree *)  
let leaf x = Tr(x, [])
```

- L'albero con radice **n** e sottoalberi **t1,...,tn** è rappresentato da **Tr(n,[t1;...;tn])**

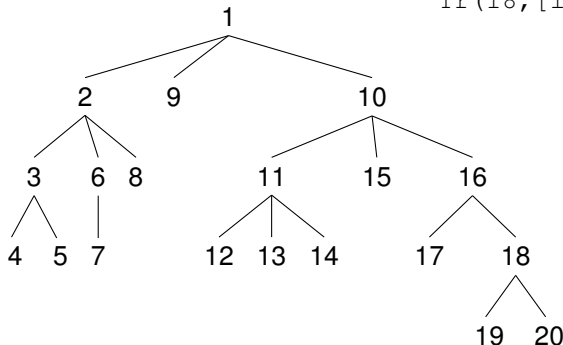
La definizione è **mutuamente ricorsiva**:

- **'a ntree** definito in termini di **'a ntree list**
- **'a ntree list** definito in termini di **'a ntree**

Esiste l'albero vuoto?

Esempio

```
let t = Tr(1,[Tr(2,[Tr(3,[leaf 4; leaf 5]);  
                Tr(6,[leaf 7]);  
                leaf 8]));  
        leaf 9;  
        Tr(10,[Tr(11,[leaf 12; leaf 13; leaf 14]);  
              leaf 15;  
              Tr(16,[leaf 17;  
                    Tr(18,[leaf 19; leaf 20])])])])])
```

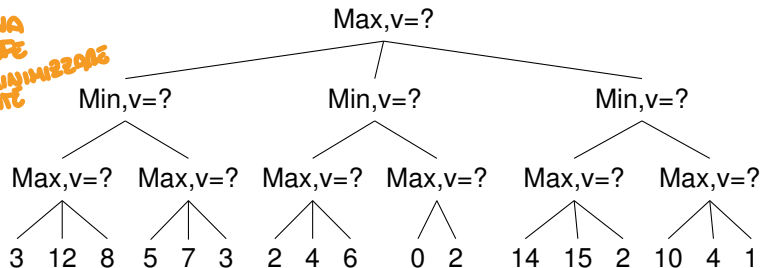


Alberi di minimax: esempio

A ogni foglia è assegnato un valore (utilità per Max della situazione corrispondente).

Il giocatore Min cerca di minimizzare l'utilità, Max di massimizzarla.

*DATA UNA
CANDIDATE
VORREI MINIMIZZARE
LE PERDITE*



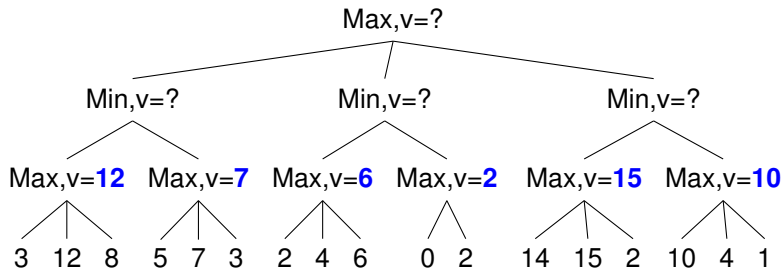
I valori si propagano verso l'alto

- Max massimizza il valore dei figli

Alberi di minimax: esempio

A ogni foglia è assegnato un valore (utilità per Max della situazione corrispondente).

Il giocatore Min cerca di minimizzare l'utilità, Max di massimizzarla.



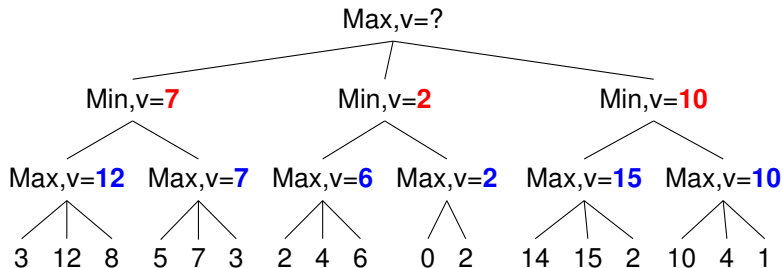
I valori si propagano verso l'alto

- Max massimizza il valore dei figli
- Min minimizza il valore dei figli

Alberi di minimax: esempio

A ogni foglia è assegnato un valore (utilità per Max della situazione corrispondente).

Il giocatore Min cerca di minimizzare l'utilità, Max di massimizzarla.



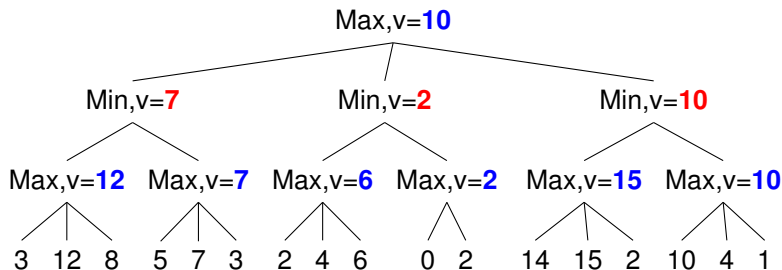
I valori si propagano verso l'alto

- Max massimizza il valore dei figli
- Min minimizza il valore dei figli
- Max massimizza il valore dei figli

Alberi di minimax: esempio

A ogni foglia è assegnato un valore (utilità per Max della situazione corrispondente).

Il giocatore Min cerca di minimizzare l'utilità, Max di massimizzarla.



I valori si propagano verso l'alto

- Max massimizza il valore dei figli
- Min minimizza il valore dei figli
- Max massimizza il valore dei figli: sceglierà la mossa di destra

Dal compito d'esame di Luglio 2013

Un albero di minimax è un albero n-ario le cui foglie sono etichettate da interi, e i nodi intermedi sono etichettati da una coppia di elementi, costituita da uno dei due valori *Min* o *Max* e da un intero.

L'intero è detto **valore numerico del nodo** (intermedio o foglia).

Rappresentiamo gli alberi di minimax mediante le seguenti dichiarazioni di tipo:

```
type player = Min | Max
type minmaxtree = Leaf of int
                | Node of (player * int) * minmaxtree list
```

Alberi di minimax (II)

I valori numerici dei nodi di un albero di minimax sono **ben assegnati** se:

- per ogni nodo intermedio etichettato da (Max, n) , n è il massimo valore numerico dei figli del nodo;
- per ogni nodo intermedio etichettato da (Min, n) , n è il minimo valore numerico dei figli del nodo.

(le foglie possono avere qualsiasi valore numerico).

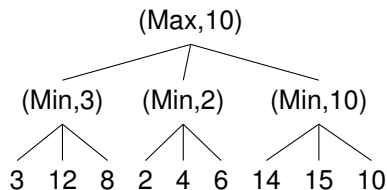
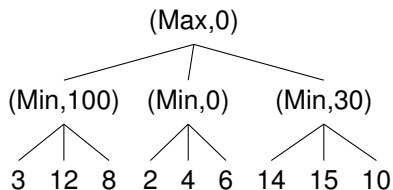
Scrivere un programma con una funzione

propagate: minmaxtree -> minmaxtree,

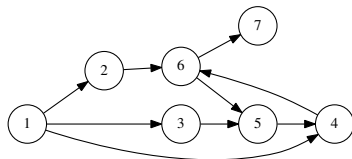
che, applicata a un qualsiasi albero T di minimax, restituisca l'albero che si ottiene da T sostituendo (eventualmente) i valori dei nodi intermedi in modo che siano ben assegnati.

Esempio

Dall'albero rappresentato a sinistra (i cui valori non sono ben assegnati) si otterrà quello rappresentato a destra:



[Vai al codice](#)



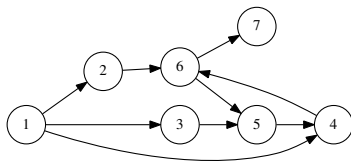
Rappresentazione mediante liste di successori

```
type 'a graph = ('a * 'a list) list
let grafo = [(1,[2;3;4]); (2,[6]); (3,[5]); (4,[6]);
             (5,[4]); (6,[5;7])]
```

Problema di base nei grafi: dato un nodo e un grafo, trovare i successori del nodo dato. Se il nodo non è nel grafo, sollevare un'eccezione

```
exception Nodo_inesistente
(* successori: 'a -> 'a graph -> 'a list *)
let successori x grafo =
  try List.assoc x grafo
  with Not_found -> raise Nodo_inesistente
```

Rappresentazione mediante lista di archi



```
type 'a graph = ('a * 'a) list
let grafo = [(1,2); (1,3); (1,4); (2,6); (3,5); (4,6);
             (5,4); (6,5); (6,7)]
```

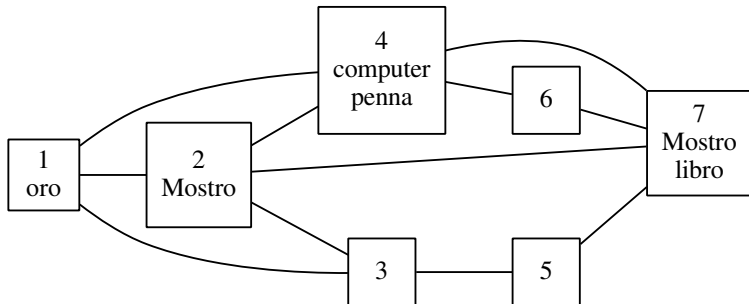
La rappresentazione è la stessa per grafi orientati e non orientati.

La differenza tra i due tipi di grafo sarà nella definizione della funzione successori (o vicini)

Vai al codice

Attraversamento di un labirinto

Problema: attraversamento di un labirinto, da una casella di entrata a una casella di uscita, senza passare per caselle che contengono un mostro, e raccogliendo tutti gli oggetti che si trovano nelle altre caselle (se ce ne sono)



[Vai al codice](#)

Una città è costituita da un insieme di posti, collegati tra loro, alcuni dei quali sono negozi che possono vendere degli oggetti.

Rappresentiamo una città mediante il tipo **city** così definito:

```
type shops = (int * string list) list
type city = (int * int) list * shops
```

(gli oggetti sono rappresentati da stringhe).

Scrivere un programma con una funzione

compra: string list -> city -> int -> int list

che, data una lista *senza ripetizioni* di stringhe **tobuy**, una città **c** e un intero **start**, riporti, se esiste, un cammino senza cicli in **c** che a partire da **start** contenga negozi nei quali è possibile comprare tutti gli oggetti contenuti in **tobuy**.

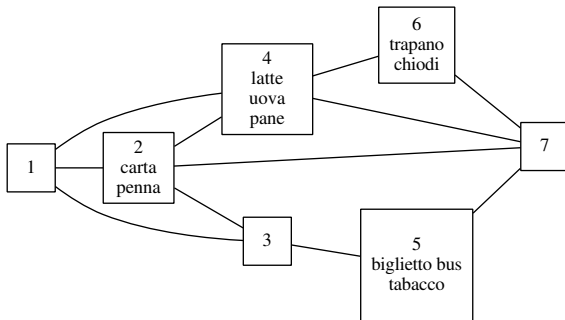
Esempio

city=(grafo,shops), dove:

```
let grafo = [(1,2); (1,3); (1,4); (2,3); (2,4); (2,7);  
            (3,5); (4,6); (4,7); (5,7); (6,7)]
```

```
let negozi = [(2, ["carta"; "penna"]);  
              (4, ["latte"; "uova"; "pane"]);  
              (5, ["biglietto bus"; "tabacco"]);  
              (6, ["trapano"; "chiodi"])]
```

(i luoghi identificati da 1, 3 e 7 non sono negozi).



Esempio (II)

Ad esempio, si può avere:

```
# compra ["chiodi"; "tabacco"; "uova"; "penna"; "pane"]  
      (grafo, negozi) 1;;  
- : int list = [1; 2; 3; 5; 7; 4; 6]
```

(dove **grafo** e **negozi** sono definiti come sopra).

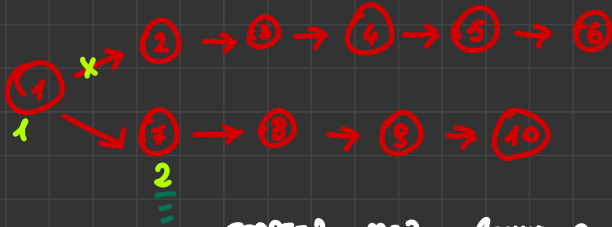
Potrebbe essere utile utilizzare la funzione **diff** così definita:

```
let diff lst1 lst2 =  
  List.filter  
    (function x -> not (List.mem x lst1)) lst2
```

per la quale era richiesto in un punto precedente dello stesso compito di determinare il tipo e dare una specifica dichiarativa.

[Vai al codice](#)

2



START=2 m=3 COUNT=0

VISITATI=2 PENDENTI=3 COUNT=0

VISITATI=2,3 PENDENTI=4 COUNT=1

VISITATI=2,3,4 PENDENTI=5 COUNT=1

VISITATI=2,3,4,5 PENDENTI=6 COUNT=2

START=1 m=3 COUNT=0

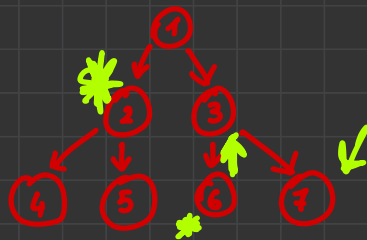
VISITATI=1 PENDENTI=2,7 COUNT=1

VISITATI=1,2 PENDENTI=3,7 COUNT=1

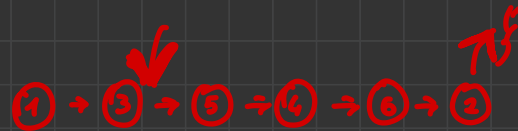
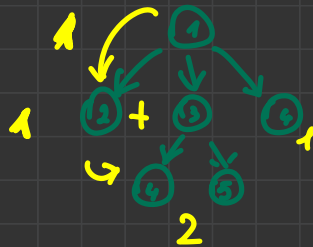
VISITATI=1,2,3 PENDENTI=4,7 COUNT=2

VISITATI=1,2,3,4 PENDENTI=5,7 COUNT=2

VISITATI=1,2,3,4,5 PENDENTI=6,7 COUNT=3



13

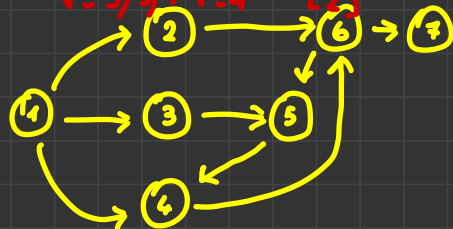


V: [] P: 1 [2,3] V: 4,5,3,1 P: 6 [2]

V: 1 P: 3 [2,3] V: 6,4,5,3,1 P: 2 [2]

V: 3,1 P: 5 [2]

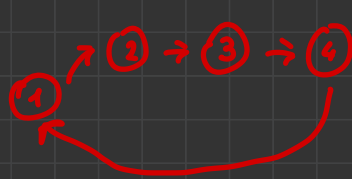
V: 5,3,1 P: 4 [2]



V: 1 P: 2,3,4 L: 2,6,3

V: 2,1 P: 6,3,4 L: 6,3

V: 6,2,1 P: 5,3,3,4



3
3 4
4,3 1
1,4,3 2
3 2,1,4,3 3

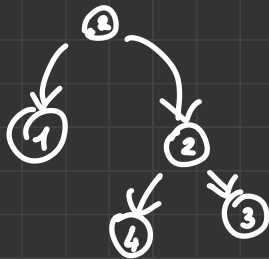
3x4

3+3+3+3

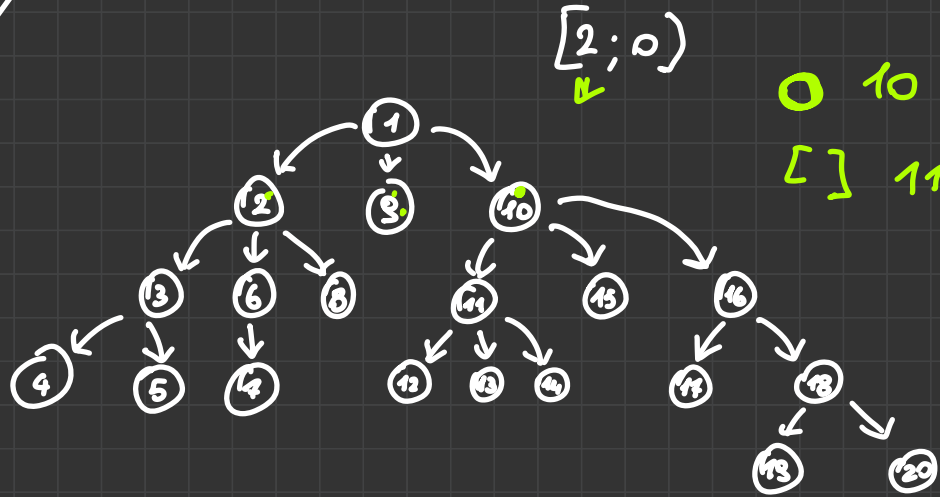
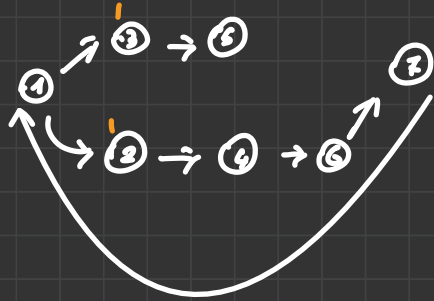
k=2 2x4 =

now exists

↓
[1 [1] [2 [4 [1, 3 []]]] }



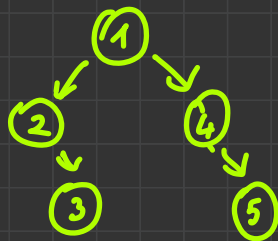
;1;6;4;2;1 []

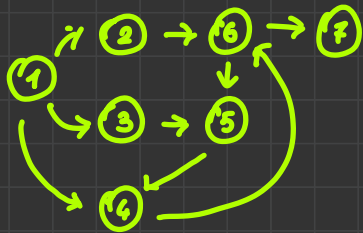




2 [3, 11] 2 11

2 3 [4; 11]





2 5

1 2 6 5